

Bitácora Técnica: Sistema Embebido IoT para Monitoreo y Control Automático de un Cultivo de Perejil

Karen C. Villaronga F.
Facultad de Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia
karencelia.villaronga@ustabuca.edu.co

Joseph S. Suarez B.
Facultad de Ingeniería Mecatrónica
Universidad Santo Tomás
Bucaramanga, Colombia
josephsantiago.suarez@ustabuca.edu.co

Abstract—This technical report documents the design, implementation, and validation of an IoT embedded system for the automated monitoring and control of a parsley (*Petroselinum crispum*) seedling prototype. Developed as the final project for the Embedded Systems course at Universidad Santo Tomás – Seccional Bucaramanga, the system integrates an ESP32-WROOM microcontroller, a DHT11 temperature and relative humidity sensor, a YL-100 soil moisture sensor, a single-channel relay-controlled drip irrigation pump, and an MG995 servo motor driving a mobile roof panel for solar radiation control. Sensor readings are published every 15 seconds to the ThingSpeak IoT platform via the MQTT protocol using the PubSubClient library, enabling real-time remote visualization across five dashboard channels. A non-blocking firmware architecture based on `millis()` timers manages concurrent tasks including sensor acquisition, hysteresis-based irrigation control, roof actuation, OLED display update, and MQTT publication without any blocking delays. Experimental results confirmed stable Wi-Fi and MQTT connectivity, correct hysteresis behavior for both the pump and the servo, and continuous data streaming to ThingSpeak at the required update rate. The prototype was built on a triangular portable container complying with the physical constraints defined in Client Request Act 1. The report also covers theoretical foundations, structured programming workshops, communication protocols, and a comparative analysis between conventional internet and IoT architectures.

Index Terms—embedded systems, IoT, MQTT, ThingSpeak, ESP32, drip irrigation, precision agriculture.

RESUMEN

Este informe técnico documenta el diseño, implementación y validación de un sistema embebido IoT para el monitoreo y control automático de un semillero portátil de perejil (*Petroselinum crispum*). Desarrollado como proyecto final del curso de Sistemas Embebidos en la Universidad Santo Tomás – Seccional Bucaramanga, el sistema integra un microcontrolador ESP32-WROOM, los sensores DHT11 (temperatura y humedad relativa del aire) y YL-100 (humedad del sustrato), una bomba de riego por goteo controlada mediante relé, y un servo motor MG995 para el accionamiento de un techo móvil de protección solar. Los datos se publican cada 15 segundos en la plataforma ThingSpeak mediante el protocolo MQTT, permitiendo la visualización remota en tiempo

real de cinco variables del sistema. El firmware implementa una arquitectura no bloqueante basada en temporizadores `millis()`, que gestiona concurrentemente la adquisición de sensores, el control con histéresis del riego y del techo, la actualización de una pantalla OLED y la publicación IoT. El informe incluye, además, fundamentos teóricos, talleres de programación estructurada, protocolos de comunicación y un análisis comparativo entre el internet convencional y la arquitectura IoT.

PALABRAS CLAVE

Sistemas embebidos, IoT, MQTT, ThingSpeak, ESP32, riego por goteo, agricultura de precisión.

I. INTRODUCCIÓN

Los sistemas embebidos constituyen uno de los pilares tecnológicos de la Industria 4.0, al integrar en un único dispositivo la capacidad de sensar el entorno, procesar información localmente y comunicarse con plataformas digitales de forma autónoma [1]. Su aplicación en contextos de agricultura de precisión —donde la gestión eficiente del agua, la temperatura y las condiciones del sustrato es determinante para la productividad— representa un caso de uso relevante y de creciente interés en ingeniería mecatrónica.

El presente documento constituye la bitácora técnica del Grupo G5 para el curso de Sistemas Embebidos de Ingeniería Mecatrónica de la Universidad Santo Tomás – Seccional Bucaramanga. El trabajo integra los contenidos desarrollados durante el semestre en tres componentes principales: un marco referencial sobre arquitecturas de firmware y multitarea en tiempo real, el desarrollo de talleres de programación estructurada aplicados a sistemas de control de acceso, y el diseño e implementación de un sistema embebido IoT para

el monitoreo y control automático de un semillero portátil de perejil (*Petroselinum crispum*).

El proyecto final consiste en un prototipo de contenedor triangular portable que integra el microcontrolador ESP32-WROOM, los sensores DHT11 y YL-100 para la medición de temperatura, humedad del aire y humedad del sustrato, un sistema de riego por goteo controlado por relé, y un techo móvil accionado por un servo motor MG995. La transmisión de datos hacia la plataforma ThingSpeak se realiza mediante el protocolo MQTT, permitiendo la visualización remota en tiempo real de las variables del sistema.

Desde el punto de vista del firmware, el sistema adopta una arquitectura no bloqueante basada en temporizadores de software (`millis()`), que permite gestionar de forma concurrente múltiples tareas sin depender de retardos bloqueantes ni de un sistema operativo de tiempo real [2]. Esta decisión de diseño es coherente con las discusiones teóricas sobre las limitaciones del Super-loop y las ventajas de la planificación por prioridades abordadas en el marco referencial y el taller integrador del curso.

El documento se organiza de la siguiente manera: la Sección II presenta el marco referencial sobre arquitecturas de firmware y tiempo real; la Sección III describe los retos del taller de programación estructurada; las Secciones IV y V abordan los protocolos de comunicación y la comparativa entre internet convencional e IoT; la Sección VI detalla el diseño e implementación del proyecto final; y las Secciones XII y XIII presentan los resultados globales y las conclusiones del trabajo.

Diversos trabajos describen técnicas de análisis espectral y aprendizaje automático aplicadas a señales. Ajusta esta sección a tu tema específico.

II. MARCO REFERENCIAL

A. Actividad 1

Para esta actividad, se requiere plantear el diseño un sistema embebido basado en un microcontrolador que gestione cinco tareas con diferentes requisitos temporales: la lectura de un sensor de alta velocidad (cada 10 ms), el control de un motor derivado de dicha lectura, el envío de telemetría vía serial (cada 100 ms), la actualización de una pantalla informativa y la atención inmediata de un botón de emergencia. El reto técnico consiste en garantizar que las tareas lentas (como la pantalla) no interfieran con las críticas (sensor y emergencia).

En la Figura 1 se observa el diagrama del problema descrito, junto con la estimación del tiempo total de ejecución del ciclo secuencial. La suma de las tareas da aproximadamente 75 ms por ciclo completo. Esto implica que, aunque el sensor debe leerse cada 10 ms, el ciclo completo tarda 75 ms, por lo que se pierden 6 de cada 7 lecturas. Asimismo, el botón de emergencia puede tardar hasta 75 ms en ser atendido, lo cual resulta inaceptable en un entorno industrial.

El reto técnico consiste en garantizar que las tareas lentas (como la pantalla) no interfieran con las críticas (sensor y emergencia).

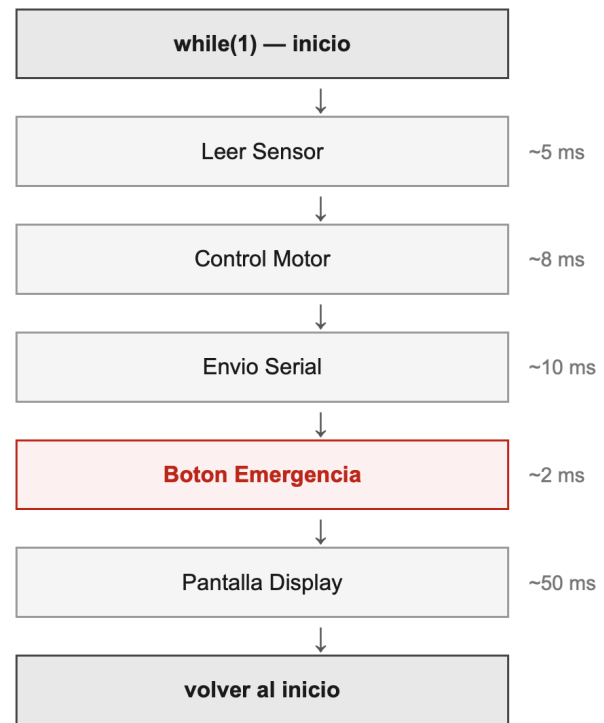


Figure 1. Diagrama del sistema secuencial con `while(1)` y tiempos acumulados por ciclo.

1) Discusión del caso

En un entorno secuencial, todo el control reside en un bucle infinito donde el procesador ejecuta una instrucción tras otra. Para gestionar los tiempos, se suelen usar banderas o contadores de tiempo (como `millis()`). El código pregunta constantemente si es hora de leer el sensor o enviar datos, ejecutando las funciones solo cuando se cumple el tiempo establecido.

Sin embargo, el mayor riesgo de esto es el bloqueo por latencia. Si la tarea de "Actualizar pantalla" tarda 30 ms en completarse, el procesador quedará atrapado en esa función, impidiendo que el sensor se lea cada 10 ms. Esto genera una

pérdida de datos y un control del motor errático. Además, el sistema se vuelve rígido, pues cualquier cambio en una tarea afecta el tiempo de todas las demás.

En esta tarea, la prioridad absoluta es el botón de emergencia, la cual es una medida de seguridad, por lo que no puede esperar a que un bucle termine su vuelta. Si el sistema está ocupado enviando un mensaje largo por el puerto serial y ocurre un accidente, un retraso de milisegundos en la detección del botón podría tener consecuencias catastróficas para el hardware o el operador [3].

Para asegurar el cumplimiento de los tiempos, es necesario transitar hacia una arquitectura orientada a eventos. Esto implica usar interrupciones para el botón de emergencia y los timers del sensor. Un RTOS facilitaría esto asignando prioridades, permitiendo que las tareas urgentes "expulsen" a las tareas lentas (como la pantalla) del CPU cuando sea necesario [2] [3].

2) Solución propuesta

La solución propuesta consiste en migrar de la arquitectura Super-loop a un RTOS (como FreeRTOS), donde cada tarea se ejecuta de forma independiente con su propio periodo y nivel de prioridad asignado por un Scheduler [2] [3]. El botón de emergencia deja de ser una línea más dentro del bucle y se convierte en una interrupción hardware (ISR, del inglés, "Interrupt Service Routine" o Rutina de Servicio de Interrupción), garantizando una respuesta en microsegundos sin importar qué esté haciendo el sistema [2]. El sensor se ejecuta como tarea periódica estricta cada 10 ms, el control del motor reacciona inmediatamente tras cada lectura, el envío serial corre cada 100 ms de forma independiente, y el display opera como tarea de fondo con la menor prioridad, de modo que sus 50 ms de ejecución nunca bloquean las tareas críticas.

Así, el Scheduler se encarga de ceder y recuperar el CPU entre tareas según su urgencia, eliminando el problema central del Super-loop: que el tiempo de respuesta de cualquier evento dependa del tiempo de ciclo total del sistema.

3) Respuesta Central: ¿Por qué no es suficiente un `while(1)`?

Programar todo en un `while(1)` (conocido como arquitectura Super-loop) no es suficiente para sistemas complejos por tres razones críticas [2].

En primer lugar, existe una falta de determinismo: el tiempo que tarda una vuelta completa del bucle depende de

qué condiciones `if` se ejecutaron. Si la escritura en pantalla tarda 50 ms, la lectura del sensor, que debe realizarse cada 10 ms, se retrasará inevitablemente.

En segundo lugar, se produce una latencia inaceptable. Los eventos urgentes deben esperar a que el puntero del programa llegue a la línea donde se evalúa la condición correspondiente. En entornos de seguridad industrial, un retardo de 100 ms puede ser suficiente para provocar un accidente.

Por último, se puede destacar que la arquitectura Super-loop es inescalable. A medida que se agregan funcionalidades, el código se vuelve frágil, por lo que modificar un simple `delay()` en una función puede comprometer el tiempo de respuesta de todo el sistema [2].

En la Figura 2 se presenta el diagrama de la solución propuesta basada en un esquema con prioridades e interrupciones. El botón de emergencia se atiende mediante una interrupción hardware de máxima prioridad (ISR < 1 μ s), garantizando respuesta inmediata. La lectura del sensor se ejecuta como tarea periódica cada 10 ms, seguida del control del motor como tarea reactiva. El envío serial se programa cada 100 ms, mientras que la actualización de la pantalla se ejecuta como tarea de fondo cuando el procesador está libre. Este enfoque asegura que las tareas críticas no sean bloqueadas por las de menor prioridad.

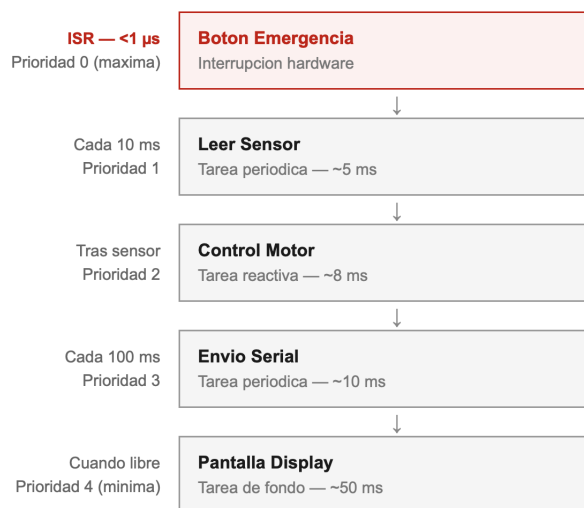


Figure 2. Arquitectura de solución basada en prioridades e interrupciones.

4) Material de consulta

a) Interrupciones

- ¿Qué son?: Son mecanismos de hardware que permite al procesador suspender temporalmente la ejecución del

programa principal para atender un evento urgente y específico [1].

- ¿Cómo funcionan en un microcontrolador?: Actúa cuando ocurre un evento, el hardware completa la instrucción actual, guarda el estado del CPU en la pila (stack) y salta a una dirección de memoria fija llamada Vector de Interrupción, donde ejecuta la ISR (Rutina de Servicio de Interrupción). Al terminar, restaura el estado y vuelve al punto exacto donde se quedó [1].
- ¿Qué problemas resuelven?: El Polling (preguntar constantemente). Sin interrupciones, el CPU pierde tiempo preguntando "¿ya presionaron el botón?" miles de veces por segundo [1].
- ¿Qué riesgos implican?: Pueden causar corrupción de datos si no se manejan variables atómicas y, si son muy frecuentes, pueden provocar que el programa principal nunca se ejecute [1].

b) Planificación y Scheduler

- ¿Qué es un scheduler?: Es el componente del núcleo (Kernel) encargado de decidir qué tarea (hilo) tiene el control del CPU en un momento dado [4].
- ¿Qué significa prioridad?: Es un valor numérico asignado a cada tarea. El Scheduler siempre favorecerá la ejecución de la tarea con mayor prioridad [4].
- ¿Qué es planificación preventiva y cooperativa?:
 - Preventiva: El Scheduler puede "quitarle" el CPU a una tarea a la fuerza si aparece una de mayor prioridad. Es ideal para tiempo real.
 - Cooperativa: Una tarea debe terminar o "ceder" voluntariamente el CPU para que otra pueda entrar. Si una tarea se queda en un bucle infinito, bloquea todo el sistema [4].

c) Multitarea

- ¿Qué es concurrencia?: Es la capacidad del sistema para manejar múltiples tareas que parecen progresar simultáneamente, aunque en un solo núcleo se ejecuten una por una intercaladas [1] [5].
- Diferencia entre multitarea real y simulada:
 - Real: Ocurre en procesadores de varios núcleos (Multi-core), donde cada núcleo ejecuta una tarea distinta

físicamente al mismo tiempo.

- Simulada: Ocurre en microcontroladores de un solo núcleo mediante el Cambio de Contexto (Context Switching), alternando tareas tan rápido que el ojo humano no percibe la pausa [5] [3].
- Ejemplos en sistemas embebidos:
 - Un sistema con RTOS como FreeRTOS donde:
 - * Una tarea maneja WiFi.
 - * Otra controla motores.
 - * Otra procesa datos de sensores [3].
 - En un ESP32 (doble núcleo):
 - * Núcleo 0 ejecuta la pila WiFi.
 - * Núcleo 1 ejecuta la lógica principal del usuario [5].

d) Sincronización

- ¿Qué es una condición de carrera?: Ocurre cuando dos o más tareas intentan modificar un recurso compartido (como una variable global) al mismo tiempo, dejando un resultado impredecible [6].
- ¿Qué son semáforos y mutex?:
 - Mutex: Abreviatura del inglés "*mutual exclusion*" o exclusión mutua, es un "candado". Solo una tarea puede tener la llave para entrar a la sección crítica [6].
 - Semáforo: Es un "contador". Permite que un número limitado de tareas accedan a un recurso o sirve para señalar entre tarea [6].
- ¿Qué es sección crítica?: Es la parte del código que accede a ese recurso compartido y que no debe ser interrumpida por ninguna otra tarea que use el mismo recurso [1] [6].

III. DESARROLLO DE TALLERES

A. Primer taller: Programación estructurada en sistemas embebidos

Este taller integrador se centra en el diseño y desarrollo de sistemas embebidos bajo el enfoque de la programación estructurada, integrando el modelado formal del sistema con su implementación práctica. A través de tres retos aplicados, el objetivo principal es desarrollar la capacidad de estructurar soluciones embebidas desde una perspectiva ingenieril, donde

se debe justificar la selección del microcontrolador, organizar el código de forma modular y documentada, e implementar buenas prácticas en todo el desarrollo del mismo.

1) RETO 1 – Sistema de Control de Acceso

- **Análisis del problema:**

El sistema de control de acceso tiene como propósito verificar una contraseña digital ingresada mediante una combinación de botones físicos o un teclado matricial. Si la combinación es correcta, se activa una cerradura electrónica (servo motor o relé); en caso contrario, se contabilizan los intentos fallidos y, al superar el límite permitido, el sistema entra en un estado de bloqueo temporal.

- **Identificación de Entradas y Salidas**

- * Entradas del sistema:

- Botones digitales (4 pulsadores) o teclado matricial 4x4 para ingreso de contraseña.
 - Señal de reset (opcional): botón para reiniciar el sistema desde estado bloqueado.

- **Restricciones Técnicas**

- * Máximo 3 intentos fallidos antes de activar el bloqueo.
 - * Tiempo de bloqueo: 30 segundos.
 - * La contraseña debe almacenarse en memoria no volátil (EEPROM) o en código.
 - * Temporización no bloqueante mediante millis()
 - * El sistema debe retornar automáticamente al estado de espera tras el desbloqueo.

- **Variables Involucradas**

- **Flujograma del sistema**

A continuación se describe el algoritmo de comportamiento del sistema de control de acceso. El diagrama representa el flujo lógico completo desde el inicio hasta la resolución de cada evento de acceso.

- **Descripción del Algoritmo (Pseudocódigo Estructurado):**

En el Algoritmo 1 se puede observar el pseudocódigo, el cual, comienza declarando un bloque de INICIO que configura los periféricos. A continuación se evalúa si el sistema está bloqueado; en caso afirmativo, se verifica

Table I
VARIABLES DEL SISTEMA DE CONTROL DE ACCESO

Variable	Tipo	Descripción
intentosFallidos	int	Contador de intentos incorrectos (0–3)
contrasenaIngresada	String	Secuencia de teclas presionadas por el usuario
contrasenaCorrecta	String	Contraseña almacenada en firmware
sistemaBloqueado	bool	Indica si el sistema está en estado de bloqueo
tiempoBloqueo	unsigned long	Marca de tiempo de inicio del bloqueo
accesoGarantizado	bool	Indicador de acceso concedido

el temporizador y se libera al cumplirse 30 segundos. Si el sistema no está bloqueado, se espera la entrada del usuario. Cuando la contraseña está completa, se ejecuta la comparación: si es correcta, se activa el servo y el LED verde; si es incorrecta, se incrementa el contador de fallos y se evalúa si se alcanzó el límite. El proceso retorna al inicio del bucle en todos los casos.

- **Justificación técnica del microcontrolador seleccionado**

- **Microcontrolador Seleccionado:** Arduino UNO (ATmega328P): El Arduino UNO con microcontrolador ATmega328P resulta adecuado para este sistema debido a su disponibilidad, bajo costo, amplia comunidad de soporte y suficiente cantidad de pines para los periféricos requeridos[7].

- **Análisis de Pines Requeridos:** El ATmega328P dispone de 14 pines digitales (6 con PWM) y 6 analógicos, cubriendo ampliamente los 12 pines requeridos con margen de expansión.

- **Capacidad de Procesamiento**

- * CPU: ATmega328P @ 16 MHz
 - * Memoria Flash: 32 KB (programa ocupa < 5 KB)
 - * SRAM: 2 KB (suficiente para buffers de contraseña y variables de estado)
 - * EEPROM: 1 KB (almacenamiento persistente de contraseña)

- **Consumo Energético**

- * Consumo activo: 46 mA @ 5V = 230 mW
 - * LED: 20 mA c/u con resistencia limitadora de 220

Algorithm 1 Algoritmo de Control de Acceso**Inicialización:**

```

1: Inicializar pines, variables y servo en posición cerrada
2: intentosFallidos = 0
3: sistemaBloqueado = false
4: while true do
5:   if sistemaBloqueado then
6:     if millis() − tiempoBloqueo ≥ 30000 then
7:       sistemaBloqueado = false
8:       intentosFallidos = 0
9:       Apagar LED_ROJO
10:    end if
11:  else
12:    Leer tecla del teclado
13:    Agregar caracter a contrasenaIngresada
14:    if |contrasenaIngresada| = |contrasenaCorrecta| then
15:      if contrasenaIngresada = contrasenaCorrecta then
16:        Activar servo (abrir cerradura)
17:        Encender LED_VERDE durante 3 segundos
18:        Regresar servo a posicion cerrada
19:        intentosFallidos = 0
20:      else
21:        intentosFallidos = intentosFallidos + 1
22:        Hacer parpadear LED_ROJO
23:        if intentosFallidos ≥ 3 then
24:          sistemaBloqueado = true
25:          tiempoBloqueo = millis()
26:        end if
27:      end if
28:      contrasenaIngresada = ""
29:    end if
30:  end if
31: end while

```

ohms

* Servo SG90: 100–250 mA en movimiento

* Alimentación recomendada: 5V/1A (USB o adaptador)

– Componentes Auxiliares

– Esquema de Conexión

En la Figura 3 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación se mencionan las conexiones establecidas en el proyecto:

* Teclado 4x4: Filas a pines D4, D5, D6, D7 —

Table II
ANÁLISIS DE PINES REQUERIDOS

Componente	Pines Requeridos	Tipo
Teclado matricial 4x4	8 pines digitales (4 filas + 4 columnas)	Digital I/O
LED verde	1 pin digital	Digital OUTPUT
LED rojo	1 pin digital	Digital OUTPUT
Servo motor	1 pin PWM	PWM OUTPUT
Buzzer	1 pin digital	Digital OUTPUT
Total	12 pines	—

Table III
COMPONENTES ELECTRÓNICOS DEL SISTEMA

Componente	Cant.	Especificación / Función
LED verde 5mm	1	3.3V, 20mA. Indicador de acceso correcto.
LED rojo 5mm	1	3.3V, 20mA. Indicador de error o bloqueo.
Resistencia 220 Ω	2	1/4 W. Limitación de corriente para los LEDs.
Buzzer pasivo	1	5V. Generación de señal sonora de retroalimentación.
Protoboard / PCB	1	830 puntos. Montaje físico del circuito.

Columnas a pines D8, D9, D10, D11

* LED verde: Pin D12 → Resistencia 220 ohms → Ánodo LED → Cátodo → GND

* LED rojo: Pin D13 → Resistencia 220 ohms → Ánodo LED → Cátodo → GND

* Servo SG90: Cable señal a Pin D3 (PWM), VCC a 5V, GND a GND

* Buzzer: Pin D2 → Buzzer (+) → GND

* Alimentación: 5V desde USB o regulador 7805

EasyEDA.

- Código documentado

El código está organizado de forma modular separando la inicialización de hardware, la lógica de verificación, el control de la cerradura y la gestión de estados de bloqueo. Se evita el uso de `delay()` para garantizar temporización

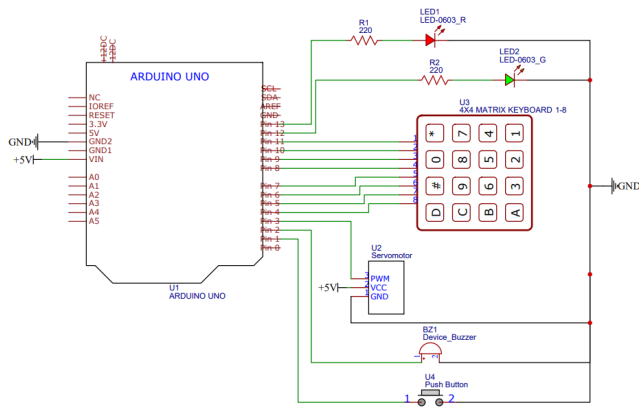


Figure 3. Esquemático de Conexión Reto 1

no bloqueante.

Listing 1. Implementación de Control de Acceso con Teclado y Servo

```
#include <Keypad.h>
#include <Servo.h>

// --- Definicion de pines ---
#define PIN_SERVO      3
#define PIN_BUZZER     2
#define PIN_LED_VERDE  12
#define PIN_LED_ROJO   13
#define PIN_BOTON      A0

// --- Configuracion del teclado 4x4 ---
const byte FILAS      = 4;
const byte COLUMNAS    = 4;

char teclas[FILAS][COLUMNAS] = {
  { '1', '2', '3', 'A' },
  { '4', '5', '6', 'B' },
  { '7', '8', '9', 'C' },
  { '*', '0', '#', 'D' }
};

byte pinesFilas[FILAS] =
  { 11, 10, 9, 8 };
byte pinesColumnas[COLUMNAS] =
  { 7, 6, 5, 4 };

Keypad teclado =
  Keypad(makeKeymap( teclas ),
    pinesFilas ,
    pinesColumnas ,
    FILAS ,
    COLUMNAS);

// --- Servo ---
Servo servo;
#define ANGULO_ABIERTO  90
#define ANGULO_CERRADO  0

// --- Variables de control de
// acceso ---
const String PASSWORD = "1234";
const byte MAX_INTENTOS = 3;
const unsigned long TIEMPO_BLOQUEO
= 30000;

// --- Estado general ---
String entradaActual = "";
byte intentosFallidos = 0;
bool bloqueado = false;
unsigned long tiempoBloqueo = 0;

// --- Servo temporizado ---
bool puertaAbierta = false;
unsigned long tiempoServo = 0;

// --- Buzzer no bloqueante ---
bool buzzerActivo = false;
unsigned long tiempoBeep = 0;
int duracionBeep = 0;

// --- Parpadeo LED rojo ---
unsigned long tiempoBlink = 0;
bool estadoBlink = false;
byte parpadeosPendientes = 0;
bool ledRojoFijo = false;

// --- Anti-rebote boton ---
unsigned long ultimoDebounce = 0;
const unsigned long
DEBOUNCE_DELAY = 50;

// =====
void inicializarHardware() {
  pinMode(PIN_LED_VERDE, OUTPUT);
  pinMode(PIN_LED_ROJO, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
}
```

```

pinMode(PIN_BOTON,
INPUT_PULLUP);
servo.attach(PIN_SERVO);
servo.write(ANGULO_CERRADO);
digitalWrite(PIN_LED_VERDE, LOW);
digitalWrite(PIN_LED_ROJO, LOW);
Serial.println(" Sistema de
acceso listo");
}

// =====
void beep(int frecuencia ,
          int duracion) {
    tone(PIN_BUZZER, frecuencia);
    buzzerActivo = true;
    tiempoBeep = millis();
    duracionBeep = duracion;
}

// =====
void actualizarBuzzer() {
    if (buzzerActivo) {
        if (millis() - tiempoBeep
            >= duracionBeep) {
            noTone(PIN_BUZZER);
            buzzerActivo = false;
        }
    }
}

// =====
void actualizarServo() {
    if (puertaAbierta) {
        if (millis() - tiempoServo
            >= 3000) {
            servo.write(ANGULO_CERRADO);
            digitalWrite(PIN_LED_VERDE,
LOW);
            puertaAbierta = false;
        }
    }
}

// =====
void actualizarBlink() {
    if (!estadoBlink) return;

```

```

    if (millis() - tiempoBlink < 300)
        return;
    tiempoBlink = millis();
    bool ledActual =
        digitalRead(PIN_LED_ROJO);
    if (!ledActual) {
        digitalWrite(PIN_LED_ROJO, HIGH);
        parpadeosPendientes--;
    } else {
        if (parpadeosPendientes == 0) {
            estadoBlink = false;
            digitalWrite(PIN_LED_ROJO,
                ledRojoFijo ? HIGH : LOW);
        } else {
            digitalWrite(PIN_LED_ROJO, LOW);
        }
    }
}

// =====
void accesoAprobado() {
    intentosFallidos = 0;
    estadoBlink = false;
    ledRojoFijo = false;
    digitalWrite(PIN_LED_ROJO, LOW);
    digitalWrite(PIN_LED_VERDE, HIGH);
    servo.write(ANGULO_ABIERTO);
    puertaAbierta = true;
    tiempoServo = millis();
    beep(2000, 150);
}

// =====
void accesoNegado() {
    intentosFallidos++;
    if (intentosFallidos <
MAX_INTENTOS)
    {
        parpadeosPendientes = 2;
        ledRojoFijo = false;
    } else {
        parpadeosPendientes = 3;
        ledRojoFijo = true;
    }
    estadoBlink = true;
    tiempoBlink = millis();

```



```

digitalWrite(PIN_LED_ROJO, LOW);
beep(500, 500);
if (intentosFallidos >=
MAX_INTENTOS)
{
    bloqueado      = true;
    tiempoBloqueo = millis();
}
}

// =====
void verificarPassword() {
    if (entradaActual == PASSWORD)
        accesoAprobado();
    else
        accesoNegado();
    entradaActual = "";
}

// =====
void procesarTecla(char tecla) {
    if (tecla == '#') {
        verificarPassword();
        return;
    }
    if (tecla == '*') {
        entradaActual = "";
        beep(1000, 100);
        return;
    }
    entradaActual += tecla;
    if (entradaActual.length()
        > PASSWORD.length()) {
        entradaActual = "";
        beep(500, 200);
    }
}

// =====
void leerBoton() {
    if (digitalRead(PIN_BOTON) == LOW) {
        if (millis() - ultimoDebounce
            > DEBOUNCE_DELAY) {
            bloqueado      = false;
            intentosFallidos = 0;
            entradaActual   = "";

```

```

estadoBlink      = false;
ledRojoFijo      = false;
digitalWrite(PIN_LED_ROJO, LOW);
digitalWrite(PIN_LED_VERDE,
LOW);
beep(1800, 200);
ultimoDebounce = millis();
}
}

// =====
void setup() {
    Serial.begin(9600);
    inicializarHardware();
}

// =====
void loop() {
    actualizarBuzzer();
    actualizarServo();
    actualizarBlink();
    leerBoton();
    if (bloqueado) {
        if (millis() - tiempoBloqueo
            >= TIEMPO_BLOQUEO) {
            bloqueado      = false;
            intentosFallidos = 0;
            estadoBlink     = false;
            ledRojoFijo     = false;
            digitalWrite(PIN_LED_ROJO, LOW);
            beep(1500, 300);
        }
        return;
    }
    char tecla = teclado.getKey();
    if (tecla) procesarTecla(tecla);
}

```

2) RETO 2 – Sistema de Control de Acceso

- Análisis del problema
 - Descripción general: El sistema de monitoreo con alarmas supervisa continuamente una variable física y clasifica su valor en tres niveles operativos: Normal, Advertencia y Crítico. Para cada nivel se activan indi-

cadores visuales y/o sonoros diferenciados. Se implementa histéresis para evitar oscilaciones inestables en los umbrales de transición.

- Identificación de Entradas y Salidas

- Entradas:

- * Sensor de temperatura (señal analógica o digital).
 - * Botón de silencio de alarma (opcional).

- Salidas:

- * LED verde: estado Normal.
 - * LED amarillo: estado Advertencia.
 - * LED rojo: estado Crítico.
 - * Buzzer: alarma sonora en Advertencia y Crítico.

- Restricciones técnicas

- * Los umbrales deben ser configurables mediante constantes.
 - * Implementación obligatoria de histéresis para evitar inestabilidad en los límites.
 - * El sistema debe funcionar en ciclo continuo con temporización no bloqueante.
 - * La lectura del sensor debe ser promediada para reducir ruido.

- Definición de umbrales y variables:

En la Tabla IV, se observa la definición de las variables con sus respectivas descripciones y justificación. Reconocer las variables a utilizar será de ayuda a la hora de realizar el código.

Table IV
DEFINICIÓN DE UMBRALES Y VARIABLES

Variable	Valor	Descripción
UMBRAL_ADVERTENCIA	const = 30°C	Umbral para activar advertencia
UMBRAL_CRITICO	const = 40°C	Umbral para nivel crítico
HISTERESIS	const = 2°C	Margen para evitar fluctuaciones
temperaturaActual	float	Valor del sensor en °C
estadoActual	enum	NORMAL, ADVERTENCIA, CRÍTICO
tiempoUltimaLectura	unsigned long	Timestamp de la lectura

- Flujograma del Sistema

El algoritmo, que se observa como Algoritmo 2, funciona en ciclo continuo, e implementa un sistema de

Algorithm 2 Control de Temperatura con Histéresis

```

INICIO
1: Inicializar pines, LCD, sensor
2: estadoActual ← NORMAL
3: loop PRINCIPAL (cada 500 ms)
4:   temperaturaActual ← leerSensor()
5:   if estadoActual = NORMAL then
6:     if temperaturaActual ≥ UMBRAL_ADVERTENCIA then
7:       estadoActual ← ADVERTENCIA
8:     end if
9:   else if estadoActual = ADVERTENCIA then
10:    if temperaturaActual ≥ UMBRAL_CRITICO then
11:      estadoActual ← CRITICO
12:    else if temperaturaActual < (UMBRAL_ADVERTENCIA – HISTERESIS) then
13:      estadoActual ← NORMAL
14:    end if
15:  else if estadoActual = CRITICO then
16:    if temperaturaActual < (UMBRAL_CRITICO – HISTERESIS) then
17:      estadoActual ← ADVERTENCIA
18:    end if
19:  end if
20:  actualizarIndicadores(estadoActual)
21:  mostrarEnLCD(temperaturaActual, estadoActual)
22: end loop
FIN

```

monitoreo de temperatura basado en una máquina de estados con tres niveles: NORMAL, ADVERTENCIA y CRÍTICO. Cada 500 ms se realiza una nueva lectura del sensor, se calcula la temperatura y se determina el estado del sistema aplicando histéresis. En función del estado, se activan los indicadores correspondientes.

- Justificación técnica del microcontrolador

- * Microcontrolador Seleccionado: Arduino UNO (ATmega328P)

El mismo microcontrolador del reto anterior es suficiente para este segundo reto. Para usar el DHT21, se requiere únicamente un pin digital con protocolo one-wire.

- * Análisis de Pines Requeridos:

En la Tabla V, se observan los pines solicitados para el correcto funcionamiento del programa.

- * Componentes auxiliares

El sistema emplea como elemento principal de medición un sensor de temperatura DHT21. La

Table V
ANÁLISIS DE PINES REQUERIDOS (SISTEMA DE MONITOREO)

Componente	Pin	Tipo
Sensor NTC / DHT21	A0 o D2	Analógico / Digital
LED verde	D9	Digital OUTPUT
LED amarillo	D10	Digital OUTPUT
LED rojo	D11	Digital OUTPUT
Buzzer	D8	Digital OUTPUT
Total	6 pines	—

visualización de la información se realiza mediante monitor serial en el Arduino IDE, lo que optimiza el uso de pines del microcontrolador.

Para la señalización del estado del sistema se utilizan tres LEDs de 5 mm (verde, amarillo y rojo) de 20 mA, cada uno con su respectiva resistencia limitadora de 220 ohms para protección. Además, se integra un buzzer activo de 5 V y aproximadamente 85 dB que funciona como alarma sonora en condiciones críticas, reforzando la advertencia al usuario.

* Esquema de conexión

En la Figura 4 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación se mencionan las conexiones realizadas:

- Sensor DHT21: D2 → pin DATA (resistencia pull-up 10k ohms entre DATA y VCC), VCC a 5V, GND a GND
- LED verde: D9 → 220 ohms → LED → GND
- LED amarillo: D10 → 220 ohms → LED → GND
- LED rojo: D11 → 220 ohms → LED → GND
- Buzzer: D8 → buzzer (+) → GND

EasyEDA.

– Código Documentado

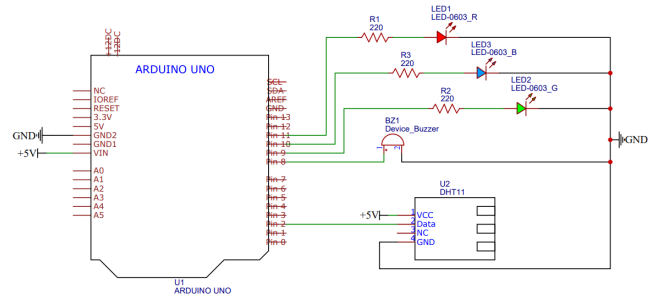


Figure 4. Esquemático de Conexión Reto 2

El código propuesto documentado se encuentra registrado a continuación:

Listing 2. Sistema de Monitoreo de Temperatura con DHT11

```
#include <DHT.h>

// --- Pines ---
const int PIN_SENSOR      = 2;
const int PIN_LED_VERDE   = 9;
const int PIN_LED_AMARILLO = 10;
const int PIN_LED_ROJO    = 11;
const int PIN_BUZZER      = 8;

// --- Configuracion DHT11 ---
#define DHTTYPE DHT11
DHT dht(PIN_SENSOR, DHTTYPE);

// --- Umbrales de temperatura ---
const float UMBRAL_ADVERTENCIA = 30.0;
const float UMBRAL_CRITICO
= 40.0;
const float HISTERESIS
= 2.0;

// --- Enumeracion de estados ---
enum EstadoSistema
{ NORMAL, ADVERTENCIA, CRITICO };
EstadoSistema estadoActual = NORMAL;

// --- Variables de temporizacion ---
unsigned long tiempoUltimaLectura = 0;
const long INTERVALO_LECTURA
= 2000;
unsigned long tiempoBuzzer
= 0;
```

```

bool estadoBuzzer
= false;

float leerTemperatura() {
    float temp = dht.readTemperature();
    if (isnan(temp)) {
        Serial.println("Error al leer
        el DHT11");
        return -999.0;
    }
    return temp;
}

void evaluarEstado(float temperatura) {
    if (temperatura == -999.0) return;
    switch (estadoActual) {
        case NORMAL:
            if (temperatura >=
                UMBRAL_ADVERTENCIA)
                estadoActual = ADVERTENCIA;
            break;
        case ADVERTENCIA:
            if (temperatura >= UMBRAL_CRITICO)
                estadoActual = CRITICO;
            else if (temperatura <
                UMBRAL_ADVERTENCIA
                - HISTERESIS)
                estadoActual = NORMAL;
            break;
        case CRITICO:
            if (temperatura < UMBRAL_CRITICO
                - HISTERESIS)
                estadoActual = ADVERTENCIA;
            break;
    }
}

void actualizarLEDs() {
    digitalWrite(PIN_LED_VERDE,
        estadoActual == NORMAL);
    digitalWrite(PIN_LED_AMARILLO,
        estadoActual ==
        ADVERTENCIA);
    digitalWrite(PIN_LED_ROJO,
        estadoActual ==
        CRITICO);
}

}

void gestionarBuzzer() {
    if (estadoActual == NORMAL) {
        noTone(PIN_BUZZER);
        return;
    }
    long intervalo =
        (estadoActual == ADVERTENCIA)
        ? 1000 : 250;
    if ((millis() - tiempoBuzzer)
        >= intervalo) {
        tiempoBuzzer = millis();
        estadoBuzzer = !estadoBuzzer;
        if (estadoBuzzer)
            tone(PIN_BUZZER, 1500,
                intervalo / 2);
        else
            noTone(PIN_BUZZER);
    }
}

void setup() {
    Serial.begin(9600);
    dht.begin();
    pinMode(PIN_LED_VERDE, OUTPUT);
    pinMode(PIN_LED_AMARILLO, OUTPUT);
    pinMode(PIN_LED_ROJO, OUTPUT);
    pinMode(PIN_BUZZER, OUTPUT);
    Serial.println("Sistema de
    Monitoreo - Activo");
}

void loop() {
    unsigned long ahora = millis();
    if (ahora - tiempoUltimaLectura >=
        INTERVALO_LECTURA) {
        tiempoUltimaLectura = ahora;
        float temperatura = leerTemperatura();
        evaluarEstado(temperatura);
        actualizarLEDs();
        Serial.print("T=");
        Serial.print(temperatura);
        Serial.print(" Estado=");
        Serial.println(estadoActual);
    }
}

```

```

    gestionarBuzzer ();
}

```

3) RETO 3 – Sistema de Control de Acceso

- **Análisis del problema**
 - Descripción general: El sistema de riego automático para cultivo de tomate monitorea continuamente la humedad relativa del ambiente mediante un sensor digital DHT11. Cuando la humedad desciende por debajo de un umbral mínimo, activa una bomba de agua (mediante relé) para iniciar el riego. El proceso se detiene al alcanzar el nivel de humedad óptimo. Se implementa histéresis para evitar ciclos de encendido/apagado rápidos y temporización no bloqueante para garantizar un funcionamiento secuencial estable. La información del sistema se visualiza en una pantalla OLED I2C de 0.96".
- **Identificación de Entradas y Salidas**
 - Entradas:
 - * Sensor DHT11: señal digital en pin D2 (humedad relativa y temperatura).
 - * Botón de riego manual (override): pin digital D3.
 - Salidas:
 - * Módulo relé 5V: activa/desactiva la bomba de agua.
 - * LED verde: humedad adecuada (IDLE).
 - * LED azul/amarillo: riego activo (REGANDO).
 - * LED rojo: humedad crítica baja (ALARMA).
 - * Pantalla OLED I2C 0.96" 128x64: visualización del porcentaje de humedad y estado del sistema.
- **Restricciones técnicas**
 - No se debe usar delay() en el ciclo principal.
 - Implementación de histéresis en umbrales de humedad.
 - El sistema debe operar de forma completamente autónoma sin intervención humana.
 - Separación de la lógica de negocio del control de hardware.
 - El DHT11 requiere un intervalo mínimo de 2000 ms entre lecturas.

- Tiempo mínimo de riego: 5 segundos para evitar microciclos.

- **Definición de umbrales y variables:**

En la Tabla VI, se observa la definición de las variables con sus respectivas descripciones y justificación. Reconocer las variables a utilizar sera de ayuda a la hora de realizar el código.

Table VI
VARIABLES INVOLUCRADAS (SISTEMA DE RIEGO AUTOMÁTICO)

Variable	Tipo	Valor Rango	/
UMBRAL_SECO	const int	30%	
UMBRAL_HUMEDO	const int	70%	
UMBRAL_ALARMA	const int	20%	
HISTERESIS	const int	5%	
humedadActual	float	0–100%	
estadoRiego	enum	IDLE / REGANDO / ALARMA	
bombaActiva	bool	true / false	
tiempoInicioRiego	unsigned long	ms	
tiempoMinRiego	const long	5000 ms	
Total	9 variables	—	

- **Flujograma del Sistema**

El sistema opera mediante un ciclo continuo con lecturas periódicas del sensor DHT11 cada 2000 ms. La lógica de control de la bomba se ejecuta basándose en el estado actual y los valores de humedad medidos, aplicando histéresis para garantizar estabilidad.

- **Justificación técnica del microcontrolador**

- Microcontrolador Seleccionado: Arduino UNO (ATmega328P)

Para el sistema de riego autónomo se selecciona el Arduino UNO. El ATmega328P es suficiente para este prototipo educativo dado que el DHT11 utiliza un único pin digital y la pantalla OLED SSD1306 se comunica por I2C, ocupando únicamente los pines A4 y A5. Para aplicaciones de campo se recomendaría el Arduino Nano o ESP32 por su menor consumo energético y posibilidad de conectividad inalámbrica.

- Análisis de Pines Requeridos:

Algorithm 3 Algoritmo de Sistema de Riego Automático**Inicialización:**

```

1: Inicializar pines, YI69, relé en OFF y pantalla OLED
2: estadoRiego = IDLE
3: bombaActiva = false
4: while true do
5:   if millis() – tiempoUltimaLectura ≥ 2000 then
6:     tiempoUltimaLectura = millis()
7:     humedadActual = dht.readHumidity()
8:     if humedadActual = invalida then
9:       Mostrar error en OLED
10:    else
11:      if estadoRiego = IDLE then
12:        if humedadActual < UMBRAL_ALARMA then
13:          estadoRiego = ALARMA
14:          activarBomba()
15:        else if humedadActual ≤ UMBRAL_SECO then
16:          estadoRiego = REGANDO
17:          activarBomba()
18:          tiempoInicioRiego = millis()
19:        end if
20:      else if estadoRiego = REGANDO then
21:        if millis() – tiempoInicioRiego ≥ tiempoMinRiego then
22:          if humedadActual ≥ UMBRAL_HUMEDO then
23:            estadoRiego = IDLE
24:            desactivarBomba()
25:          end if
26:        end if
27:      else if estadoRiego = ALARMA then
28:        activarBomba()
29:        if humedadActual ≥ (UMBRAL_SECO + HISTERESIS) then
30:          estadoRiego = IDLE
31:          desactivarBomba()
32:        end if
33:      end if
34:      actualizarIndicadores(estadoRiego)
35:      mostrarP(humedadActual, estadoRiego)
36:    end if
37:  end if
38:  gestionarBuzzerAlarma()
39:  verificarBotonManual()
40: end while

```

En la Tabla VII, se observan los pines solicitados para el correcto funcionamiento del programa.

– Capacidad de Procesamiento y Consumo

- * CPU: ATmega328P @ 16 MHz — suficiente para lectura digital y control.

Table VII
ANÁLISIS DE PINES REQUERIDOS (SISTEMA DE RIEGO AUTOMÁTICO)

Componente	Pin	Tipo	Descripción
Sensor DHT11	D2	Digital IN	Lectura de humedad relativa y temperatura
Módulo relé 5V	D4	Digital OUT	Control de bomba de agua
LED verde (IDLE)	D5	Digital OUT	Estado normal
LED azul (REGANDO)	D6	Digital OUT	Estado de riego activo
LED rojo (ALARMA)	D7	Digital OUT	Estado de alarma
Buzzer	D8	Digital OUT	Alarma sonora
Botón manual	D3	Digital IN	Detención manual del riego
OLED SSD1306 0.96"	A4/A5	I2C	Visualización del sistema
Total	8 pines	—	—

* Consumo sistema: 85 mA (sin bomba); el DHT11 consume 1 mA y la OLED 20 mA en operación.

* Relé 5V: bobina 70 mA; la bomba se alimenta de fuente externa.

* Se recomienda fuente de 12V/2A para alimentar tanto el Arduino como la bomba.

* El relé desacopla galvánicamente el microcontrolador de la carga.

– Componentes auxiliares

En la tabla VIII, se observan los componentes relevantes al proyecto a desarrollar.

– Esquema de conexión En la Figura 5 se observa el diagrama de conexión correspondiente al funcionamiento de este circuito, desarrollado a través del programa de circuitado. A continuación, se mencionan los pines conectados.

* DHT11: pin DATA a D2, VCC a 5V, GND a GND

* Relé: IN a pin D4, VCC a 5V, GND a GND; bornes COM/NO a bomba y fuente externa

Table VIII
COMPONENTES AUXILIARES (SISTEMA DE RIEGO AUTOMÁTICO)

Componente	Especificación	Función
Sensor DHT11	Digital, 3.3–5V, $\pm 5\%$ HR	Medición de humedad relativa del ambiente
Módulo relé 1 canal 5V	Carga máx. 10A / 250VAC	Conmutación de la bomba de agua
Mini bomba de agua 5V	120 L/h, 0.5–1.2W	Suministro de agua al cultivo
Tubería de silicona 6mm	—	Conducción del agua
LEDs 5mm (verde, rojo, azul)	20 mA c/u	Indicadores visuales de estado
Resistencias 220 Ω	3 unidades	Limitación de corriente para LEDs
OLED SSD1306 0.96"	I2C, 128x64 px, ~ 20 mA	Visualización de datos en tiempo real
Fuente 12V / 2A	DC regulada	Alimentación del sistema completo

- * LED verde: D5 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * LED azul: D6 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * LED rojo: D7 $\rightarrow$ 220 ohms $\rightarrow$ LED $\rightarrow$ GND
- * Buzzer activo: D8 $\rightarrow$ buzzer (+) $\rightarrow$ GND
- * Botón manual: D3 con resistencia pull-up interna $\rightarrow$ GND al presionar
- * OLED SSD1306: SDA a A4, SCL a A5, VCC a 3.3V o 5V, GND a GND

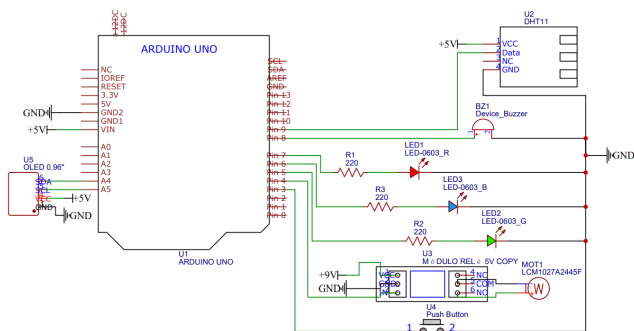
EasyEDA.

Figure 5. Esquemático de Conexión Reto 3

- Código Documentado

El código propuesto documentado se encuentra registrado a continuación:

Listing 3. Sistema de Riego Automatico con DHT11 y OLED

```
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
// --- Definicion de pines ---
const int PIN_SENSOR_HUMEDAD = 2;
const int PIN_RELE            = 4;
const int PIN_LED_VERDE      = 5;
const int PIN_LED_AZUL       = 6;
const int PIN_LED_ROJO       = 7;
const int PIN_BUZZER         = 8;
const int PIN_BOTON_MANUAL    = 3;
// --- Configuracion DHT11 ---
#define DHTTYPE DHT11
DHT dht(PIN_SENSOR_HUMEDAD, DHTTYPE);
// --- Configuracion OLED ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
Adafruit_SSD1306 oled(SCREEN_WIDTH,
                      SCREEN_HEIGHT,
                      &Wire,
                      OLED_RESET);
// --- Umbrales de humedad ---
const int UMBRAL_SECO      = 30;
const int UMBRAL_HUMEDO    = 70;
const int UMBRAL_ALARMA    = 20;
const int HISTERESIS       = 5;
// --- Temporizacion ---
const long INTERVALO_LECTURA = 2000;
const long TIEMPO_MIN_RIEGO    = 5000;
// --- Enumeracion de estados ---
enum EstadoRiego
{ IDLE, REGANDO, ALARMA };
EstadoRiego estadoRiego = IDLE;
// --- Variables de control ---
float          humedadActual
= 0.0;
bool           bombaActiva
= false;
unsigned long tiempoUltimaLectura = 0;
unsigned long tiempoInicioRiego   = 0;
unsigned long tiempoBuzzer        = 0;
```

```

bool estadoBuzzer
= false;
float leerHumedad() {
    float h = dht.readHumidity();
    if (isnan(h)) {
        Serial.println("Error DHT11");
        oled.clearDisplay();
        oled.setCursor(0, 0);
        oled.println("Error sensor");
        oled.println("DHT11 fallo");
        oled.display();
        return -1.0;
    }
    return h;
}

void activarBomba() {
    if (!bombaActiva) {
        digitalWrite(PIN_RELE, LOW);
        bombaActiva = true;
        tiempoInicioRiego = millis();
        Serial.println("BOMBA: ACTIVADA");
    }
}

void desactivarBomba() {
    if (bombaActiva) {
        digitalWrite(PIN_RELE, HIGH);
        bombaActiva = false;
        Serial.println("BOMBA: DESACTIVADA");
    }
}

void evaluarEstadoRiego() {
    bool tiempoMinCumplido =
        (millis() - tiempoInicioRiego)
        >= TIEMPO_MIN_RIEGO;
    switch (estadoRiego) {
        case IDLE:
            if (humedadActual < UMBRAL_ALARMA){
                estadoRiego = ALARMA;
                activarBomba();
            } else if (humedadActual
                <= UMBRAL_SECO) {
                estadoRiego = REGANDO;
                activarBomba();
            }
            break;
        case REGANDO:
            if (tiempoMinCumplido &&
                humedadActual >=
                    UMBRAL_HUMEDO) {
                estadoRiego = IDLE;
                desactivarBomba();
            }
            break;
        case ALARMA:
            activarBomba();
            if (humedadActual >=
                (UMBRAL_SECO + HISTERESIS)) {
                estadoRiego = IDLE;
                desactivarBomba();
            }
            break;
    }
}

void actualizarIndicadores() {
    digitalWrite(PIN_LED_VERDE,
        estadoRiego == IDLE);
    digitalWrite(PIN_LED_AZUL,
        estadoRiego == REGANDO);
    digitalWrite(PIN_LED_ROJO,
        estadoRiego == ALARMA);
}

void gestionarBuzzerAlarma() {
    if (estadoRiego != ALARMA) {
        noTone(PIN_BUZZER);
        return;
    }
    if ((millis() - tiempoBuzzer) >= 500){
        tiempoBuzzer = millis();
        estadoBuzzer = !estadoBuzzer;
        if (estadoBuzzer)
            tone(PIN_BUZZER, 880, 250);
        else
            noTone(PIN_BUZZER);
    }
}

void verificarBotonManual() {
    if (digitalRead(PIN_BOTON_MANUAL)
        == LOW) {
        delay(50);
        if (digitalRead(PIN_BOTON_MANUAL)
            == LOW) {
            desactivarBomba();

```



```

        estadoRiego = IDLE;
        Serial.println(
            "OVERRIDE: Riego detenido");
    }
}

void mostrarEnOLED() {
    oled.clearDisplay();
    oled.setTextSize(1);
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 0);
    oled.println("Sistema de Riego");
    oled.drawLine(0, 10, 127, 10,
        SSD1306_WHITE);

    oled.setTextSize(2);
    oled.setCursor(0, 16);
    oled.print("Hum: ");
    oled.print((int)humedadActual);
    oled.println("%");
    oled.setTextSize(1);
    oled.setCursor(0, 40);
    oled.print("Estado: ");
    switch (estadoRiego) {
        case IDLE:
            oled.println("OPTIMO");    break;
        case REGANDO:
            oled.println("REGANDO");    break;
        case ALARMA:
            oled.println("ALARMA!");    break;
    }
    oled.setCursor(0, 52);
    oled.print("Bomba: ");
    oled.println(bombaActiva ?
        "ON":"OFF");
    oled.display();
}

void setup() {
    Serial.begin(9600);
    dht.begin();
    pinMode(PIN_RELE,          OUTPUT);
    pinMode(PIN_LED_VERDE,     OUTPUT);
    pinMode(PIN_LED_AZUL,      OUTPUT);
    pinMode(PIN_LED_ROJO,      OUTPUT);
    pinMode(PIN_BUZZER,        OUTPUT);
    pinMode(PIN_BOTON_MANUAL,   INPUT_PULLUP);
}

```

```

digitalWrite(PIN_RELE, HIGH);
oled.begin(SSD1306_SWITCHCAPVCC,
    0x3C);
oled.clearDisplay();
oled.setTextSize(1);
oled.setTextColor(SSD1306_WHITE);
oled.setCursor(20, 20);
oled.println("Sistema Riego");
oled.setCursor(25, 35);
oled.println("Iniciando ...");
oled.display();
delay(2000);
oled.clearDisplay();
Serial.println(
    "Sistema de Riego - Listo");
}

void loop() {
    unsigned long ahora = millis();
    if (ahora - tiempoUltimaLectura
        >= INTERVALO_LECTURA) {
        tiempoUltimaLectura = ahora;
        humedadActual = leerHumedad();
        if (humedadActual < 0) return;
        evaluarEstadoRiego();
        actualizarIndicadores();
        mostrarEnOLED();
        Serial.print("Humedad: ");
        Serial.print(humedadActual);
        Serial.print("% | Estado: ");
        Serial.println(estadoRiego);
    }
    gestionarBuzzerAlarma();
    verificarBotonManual();
}

```

IV. PROTOCOLOS DE COMUNICACIÓN

- ¿Por qué no basta con "tener Wi-Fi"? Una solución IoT completa involucra múltiples capas de comunicación. Algunas se encargan de conectar sensores al microcontrolador, otras permiten enlazar el nodo con internet, y otras definen cómo se intercambian los datos a nivel de aplicación. Reducir el IoT únicamente a la conectividad Wi-Fi constituye un error conceptual frecuente, ya que ignora la complejidad real de los sistemas embebidos conectados.

- Comunicación interna del nodo embebido (dentro del hardware) Antes de transmitir información a la red, el sistema debe capturar y procesar correctamente los datos provenientes de los sensores:

- * GPIO y señales digitales: lectura de estados y activación de actuadores.
- * Entradas analógicas y ADC: adquisición de variables continuas como temperatura o presión.
- * PWM: control de velocidad, intensidad luminosa o posición.
- * I2C: comunicación con múltiples sensores usando pocos conductores.
- * SPI: transmisión de alta velocidad con memorias, pantallas o sensores especializados.
- * UART: comunicación serial para depuración o conexión con módulos externos.

- Tecnologías inalámbricas para conectar el nodo a la red

La selección de la tecnología de comunicación depende del equilibrio entre alcance, consumo energético, velocidad de transmisión y costo de implementación. En la Tabla IV se resumen las tecnologías más utilizadas. No existe una tecnología universalmente superior; la elección depende del caso de uso específico.

- Notas específicas de cada tecnología

- * Wi-Fi: basado en el estándar 802.11, ideal para transmisión de grandes volúmenes de datos y conexión directa a internet.
- * Bluetooth / BLE: optimizado para bajo consumo, fundamental en dispositivos portátiles y sensores de batería.
- * ZigBee: diseñado para redes de sensores distribuidos que operan de forma cooperativa.
- * Thread: protocolo mallado basado en IPv6, auto-reparable y eficiente energéticamente.
- * LoRaWAN: orientado a telemetría de largo alcance con bajo consumo y baja frecuencia de transmisión.
- * Celular / NB-IoT: permite independencia de redes locales, aunque con mayor complejidad operativa.

- Protocolos de aplicación: cómo se empaqueta y trans-

porta la información

Una vez establecida la conexión de red, es necesario definir el protocolo de intercambio de datos:

- * HTTP / REST: modelo cliente-servidor basado en solicitudes y respuestas. Adecuado para servicios web y pruebas iniciales.
- * MQTT: protocolo ligero basado en publicador/suscriptor, altamente eficiente para telemetría y sistemas distribuidos. El broker gestiona la distribución de mensajes.
- * WebSocket: permite comunicación bidireccional continua mediante conexión persistente, útil para dashboards y control en tiempo real.

En aplicaciones con ESP32, MQTT resulta especialmente formativo para comprender la lógica IoT, mientras que HTTP facilita la integración con servicios web.

- Criterio de selección resumido

No existe una solución única óptima. La elección adecuada surge del análisis de los requerimientos del sistema:

- * ¿Qué alcance de comunicación se necesita?
- * ¿Qué volumen de datos se transmitirá y con qué frecuencia?
- * ¿Cuál es el consumo energético permitido?
- * ¿Cuál es el costo y la facilidad de implementación en el entorno específico?

V. IOT VS INTERNET CONVENCIONAL

- ¿Qué es el IoT y de dónde viene?

El término “Internet de las Cosas” fue acuñado por Kevin Ashton en 1999, en el contexto del MIT, para describir la idea de conectar objetos físicos a internet mediante etiquetas RFID. Su consolidación como paradigma tecnológico dependió del avance simultáneo de tres factores: el desarrollo de microcontroladores cada vez más pequeños, económicos y eficientes; la masificación de la conectividad inalámbrica; y la adopción del protocolo IPv6, que permitió asignar direcciones únicas a miles de millones de dispositivos. El cambio fundamental no fue solo conectar dispositivos, sino convertir el entorno físico en una fuente permanente de datos útiles para supervisión, decisión y acción.

Table IX
TECNOLOGÍAS INALÁMBRICAS PARA CONECTAR EL NODO A LA RED

Tecnología	Alcance	Consumo	Tasa de datos	Uso típico
Wi-Fi	~100 m	Alto	Hasta 1 Gbps	Hogares, oficinas, prototipos ESP32
Bluetooth / BLE	10–100 m	Bajo	1–3 Mbps	Wearables, sensores cercanos
ZigBee	~100 m	Bajo	250 kbps	Domótica, automatización industrial
LoRa / LPWAN	2–15 km	Muy bajo	0.3–50 kbps	Agricultura, ciudades inteligentes, campo abierto
NFC / RFID	< 10 cm	Muy bajo	424 kbps	Identificación, pagos, logística
Celular 4G/5G	Km	Variable	Hasta Gbps	Vehículos, monitoreo remoto sin red local
Ethernet	Cableado	Bajo/medio	Alta	Ambientes industriales fijos, alta estabilidad

– Diferencia conceptual entre los tres paradigmas

En el **internet tradicional**, la interacción ocurre entre personas y servicios web: el usuario consulta, descarga o accede a plataformas. Los datos son texto, imagen, audio o video, y el flujo es persona → red → servicio. La generación de datos depende completamente de la acción deliberada del usuario humano; sin interacción, no hay dato.

En el **IoT**, los objetos físicos captan, procesan y comunican datos de forma autónoma. El entorno físico se convierte en fuente permanente de información sin requerir intervención humana directa. El flujo cambia a: sensor → procesamiento → red → decisión → acción. Los datos dominantes son variables físicas del entorno como temperatura, humedad, presión o posición. Los objetivos son el monitoreo, control y automatización distribuida. Ejemplos representativos incluyen estaciones ambientales, casas inteligentes y sistemas de riego automatizado.

En el **IIoT** (Internet Industrial de las Cosas), los nodos se integran con máquinas, procesos industriales, líneas de producción, inventarios y sistemas de trazabilidad. Los datos dominantes son variables de proceso, estado de activos, calidad y trazabilidad productiva. El objetivo central es la optimización operativa, el mantenimiento predictivo, la eficiencia energética y la producción inteligente. A diferencia del IoT doméstico o académico, el IIoT opera bajo requisitos más estrictos de confiabilidad, latencia y seguridad.

En la **Industria 4.0**, el dato deja de ser solo un registro histórico y se convierte en base para analítica avanzada, interoperabilidad entre sistemas heterogéneos, decisión en tiempo real y automatización inteligente apoyada en inteligencia artificial y gemelos digitales.

– IoT desde la perspectiva de sistemas embebidos

Un sistema embebido en el contexto IoT deja de ser un dispositivo aislado y se convierte en un nodo activo dentro de una arquitectura mayor. Para cumplir este rol, debe integrar cinco funciones fundamentales:

- * **Sensar:** captar variables físicas del entorno mediante sensores de temperatura, humedad, presión, corriente, vibración, posición, entre otros.
- * **Procesar:** acondicionar, filtrar, escalar, convertir y organizar los datos dentro del microcontrolador o en una etapa de procesamiento en el borde (*edge computing*).
- * **Comunicar:** transmitir la información hacia otros nodos o hacia internet mediante un medio físico y un protocolo de aplicación apropiados al contexto.
- * **Visualizar o almacenar:** presentar los datos al usuario final a través de dashboards o interfaces web, o registrarlos en bases de datos para análisis posterior.
- * **Actuar:** ejecutar decisiones que modifiquen el proceso físico mediante relés, válvulas, motores, alarmas o consignas de control.

La idea clave es que el microcontrolador no es el sistema completo, sino únicamente el nodo local. El valor real del IoT aparece cuando el dato sale del dispositivo, viaja por la red, se integra con información de otros nodos y se convierte en conocimiento útil para tomar decisiones concretas.

– Arquitectura por capas de un sistema IoT

Los sistemas IoT se organizan típicamente en una arquitectura de cuatro capas, cada una con una función

específica dentro del flujo del dato:

- * **Capa física o de percepción:** contiene los sensores, actuadores, el microcontrolador, conversores ADC y pines GPIO. Es el punto de contacto directo con el mundo físico. Pregunta de diseño: ¿qué se mide, con qué precisión, con qué frecuencia de muestreo y qué variables se controlan?
- * **Capa de comunicación:** se encarga de mover la información desde la capa física hacia internet o hacia otros nodos. Incluye tecnologías como Wi-Fi, BLE, Zigbee, LoRa, Ethernet, redes móviles y gateways intermediarios. Pregunta de diseño: ¿qué alcance se necesita, qué consumo energético es aceptable, qué protocolo conviene según el volumen y frecuencia de datos?
- * **Capa de servicio:** gestiona la recepción, almacenamiento, procesamiento y protección de los datos. Incluye brokers MQTT, bases de datos relacionales o de series temporales, servidores web, APIs REST, dashboards de visualización, reglas de negocio y sistemas de alertas. Pregunta de diseño: ¿dónde se almacenan los datos, cómo se visualizan en tiempo real, qué nivel de seguridad y autenticación se requiere?
- * **Capa de aplicación:** es la que entrega valor directo al usuario o al proceso. Habilita funciones de monitoreo remoto, control automatizado, predicción de fallos, mantenimiento preventivo, trazabilidad de activos y optimización logística. Pregunta de diseño: ¿qué decisión concreta permite tomar el sistema, qué indicador operativo mejora?

– Transición IoT → IIoT → Industria 4.0 en síntesis

- * **IoT:** monitorea variables ambientales o de dispositivos individuales en contextos domésticos, urbanos o académicos. El alcance es limitado y los requisitos de confiabilidad son moderados.
- * **IIoT:** integra datos de múltiples nodos distribuidos para mejorar productividad, disponibilidad de activos, seguridad operacional y calidad del producto en entornos industriales exigentes.
- * **Industria 4.0:** el dato se convierte en el recurso central. Se habilitan capacidades de analítica avanzada, interoperabilidad entre sistemas heterogé-

neos, automatización inteligente y decisión en tiempo real apoyada en modelos predictivos.

– Desafíos recurrentes que no deben ignorarse

Independientemente de la escala del sistema IoT, existen desafíos transversales que deben considerarse desde las etapas iniciales del diseño:

- * **Seguridad:** proteger credenciales de acceso, cifrar los canales de comunicación y controlar el acceso a los datos almacenados. Un sistema IoT mal asegurado representa un vector de ataque crítico.
- * **Energía:** estimar el consumo de cada componente, definir la autonomía requerida y seleccionar el tipo de alimentación adecuado, especialmente en nodos remotos alimentados por batería o energía solar.
- * **Robustez:** prever y gestionar situaciones de pérdida de conexión, reinicios inesperados, errores de lectura de sensores y condiciones ambientales adversas. El sistema debe recuperarse de forma autónoma.
- * **Escalabilidad:** diseñar la arquitectura de modo que funcione correctamente con un nodo, con diez o con una red de cientos de dispositivos, sin requerir rediseños estructurales.
- * **Mantenibilidad:** documentar con claridad las conexiones físicas, las variables del sistema, los formatos de mensaje, las versiones de librerías y las dependencias de software, para facilitar el mantenimiento y la actualización futura del sistema.

VI. PROYECTO FINAL

A. Descripción del problema

En los cultivos de hortalizas a pequeña escala, el riego y el control ambiental se realizan de forma manual. En el caso específico del perejil (*Petroselinum crispum*), esta situación es crítica porque la especie es sensible tanto a la deficiencia hídrica como al exceso de temperatura, lo que afecta directamente su germinación —requiere humedad constante del sustrato entre 60 y 80 %— y su desarrollo vegetativo —temperatura óptima entre 15 y 25 °C—.

El riego manual genera ineficiencias como uso excesivo o insuficiente del agua, variabilidad en la humedad del sustrato, estrés hídrico, exposición térmica no controlada y baja

capacidad de seguimiento técnico del proceso. La ausencia de un mecanismo de protección solar automático agrava el problema durante las horas de alta irradiación.

El Grupo G5 diseñará e implementará un prototipo portable de semillero con geometría triangular equilátera, base de 40 cm y perímetro máximo de 160 cm. El sistema contará con los siguientes componentes principales:

- Tarjeta de desarrollo **ESP32-WROOM**.
- Sensor **DHT11** para medición de temperatura y humedad relativa del aire.
- Sensor **YL-100** para medición de humedad volumétrica del sustrato.
- **Sistema de riego por goteo**: bomba sumergible de 5 V con filtro de malla y tres goteros ajustables distribuidos sobre el sustrato.
- **Techo móvil** accionado por un servo-motor SG90: se cierra automáticamente cuando la temperatura supera el umbral definido (`TEMP_MAX`), protegiendo el cultivo de la irradiación solar directa, y se abre cuando la temperatura desciende por debajo de `TEMP_MAX - HYSTERESIS_T`.
- Plataforma IoT **ThingSpeak** con publicación de datos vía protocolo **MQTT** (broker `mqtt3.thingspeak.com`).
- Semilla de **perejil** (*Petroselinum crispum*), suministrada por la docente encargada del proyecto.

Problema central: El cultivo experimental de perejil carece de un sistema automático capaz de mantener condiciones óptimas de humedad del sustrato y temperatura ambiental, activar el riego por goteo de forma autónoma, controlar un techo móvil para protección solar y visualizar remotamente el estado del sistema en ThingSpeak, reduciendo la necesidad de intervención manual continua.

B. Objetivos

1) Objetivo general

Diseñar e implementar un sistema embebido IoT para el monitoreo remoto y control automático de variables asociadas al crecimiento de perejil en un prototipo triangular portable con techo móvil, integrando los sensores DHT11 y YL-100, actuadores de riego por goteo y servo-motor para el control

del techo, comunicación inalámbrica Wi-Fi mediante ESP32-WROOM y visualización remota en ThingSpeak vía MQTT.

2) Objetivos específicos

- 1) Definir los requisitos funcionales y no funcionales del sistema, considerando las variables medidas por el DHT11 y el YL-100, y los requerimientos agronómicos de germinación y crecimiento del perejil.
- 2) Diseñar la arquitectura hardware/software y el diseño físico 3D del prototipo, justificando la selección de la tarjeta ESP32-WROOM, los sensores, los actuadores, el mecanismo de techo móvil y la plataforma ThingSpeak.
- 3) Implementar el prototipo funcional sobre el contenedor triangular, incorporando lógica de control con histéresis para el riego y la apertura/cierre del techo, y comunicación remota vía MQTT hacia ThingSpeak.
- 4) Validar el sistema mediante pruebas experimentales documentadas en bitácora técnica y repositorio GitHub con historial de commits organizado.

C. Requisitos funcionales

La Tabla X presenta los requisitos funcionales identificados para el sistema, definidos mediante el código RF-XX.

D. Requisitos no funcionales

La Tabla XI presenta los requisitos no funcionales identificados para el sistema, definidos mediante el código RNF-XX.

E. Diagrama General del Sistema

1) Arquitectura de Capas

El sistema se organiza en cuatro capas funcionales. El flujo de datos es bidireccional entre el ESP32 y ThingSpeak, y se organiza como se muestra en la Figura 6.

2) Variables del Sistema

Las variables mínimas del sistema se resumen en la Tabla XII, con su tipo, sensor/actuador asociado, rango de valores y uso específico dentro del sistema.

3) Flujo de Control del Firmware

El firmware opera con un bucle principal no bloqueante (`millis()`). En cada iteración se evalúan tres temporizadores independientes:

Table X
REQUISITOS FUNCIONALES DEL SISTEMA – GRUPO G5

ID	Descripción	Prioridad	Fuente
RF-01	Leer temperatura ambiente con DHT11, mínimo 1 lectura cada 5 s.	Alta	Enunciado XI
RF-02	Leer humedad relativa del aire con DHT11, misma frecuencia que RF-01.	Alta	Enunciado XI
RF-03	Leer humedad del sustrato con YL-100 (salida analógica al ADC), mínimo 1 lectura cada 10 s.	Alta	Enunciado XI
RF-04	Activar la bomba de riego por goteo cuando humedad_sustrato < HUMIDITY_MIN.	Alta	Enunciado XIII-A
RF-05	Desactivar la bomba de riego cuando humedad_sustrato > HUMIDITY_MIN + HYSTERESIS.	Alta	Enunciado XIII-A
RF-06	Cerrar el techo móvil (servo a 90°) cuando temperatura > TEMP_MAX, para proteger el cultivo del calor y la irradiación solar directa.	Alta	Enunciado XIII-B / Acta 1
RF-07	Abrir el techo móvil (servo a 0°) cuando temperatura < TEMP_MAX – HYSTERESIS_T.	Alta	Enunciado XIII-B
RF-08	Publicar en ThingSpeak vía MQTT las 5 variables del sistema con intervalo máx. de 15 s.	Alta	Enunciado XIV / ThingSpeak API
RF-09	Cada publicación IoT debe serializar los datos en formato de campo ThingSpeak (field1...field5) compatible con mqtt3.thingspeak.com.	Alta	ThingSpeak Docs
RF-10	Implementar temporización no bloqueante con millis() — sin uso de delay() en el bucle principal.	Alta	Enunciado XVI
RF-11	Incluir filtro mecánico del agua (malla) antes de la línea de riego por goteo.	Alta	Acta 1 núm. 8
RF-12	Operar sobre contenedor triangular: base 40 cm, perímetro máx. 160 cm, altura sustrato mín. 10 cm.	Alta	Acta 1 núm. 4, 6
RF-13	Organizar firmware en módulos: readSensors(), controlIrrigation(), controlRoof(), publishThingSpeak(), updateActuators().	Media	Enunciado XVI

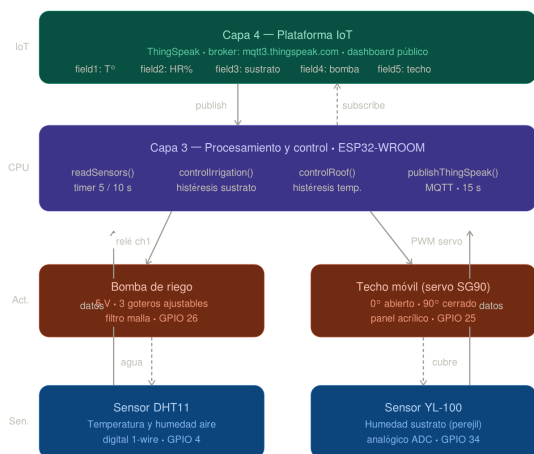


Figure 6. Arquitectura de capas del sistema

- **Timer sensores (5/10 s):** llama a `readSensors()` — lee DHT11 y YL-100, valida rangos.
- **Timer control:** llama a `controlIrrigation()` con histéresis sobre `humedad_sustrato`; luego `controlRoof()` con histéresis sobre temperatura; luego `updateActuators()` (GPIO relé + PWM servo).
- **Timer IoT (15 s):** llama a `publishThingSpeak()` — publica los 5 campos en el canal ThingSpeak vía MQTT.

Table XI
REQUISITOS NO FUNCIONALES DEL SISTEMA – GRUPO G5

ID	Descripción	Categoría	Fuente
RNF-01	Firmware en VS Code con PlatformIO. Prohibido .ino y Arduino IDE.	Restricción	Enunciado X
RNF-02	Código en GitHub con historial de commits organizado que evidencie avance progresivo.	Trazabilidad	Enunciado X
RNF-03	Operación continua mínimo 30 min sin fallos de sensor, actuador ni desconexión IoT.	Disponibilidad	Enunciado XXI
RNF-04	Prototipo portable, peso estimado menor a 5 kg en operación completa.	Portabilidad	Acta 1 núm. 3
RNF-05	Latencia máxima de 15 s desde adquisición hasta actualización de canal en ThingSpeak (restricción free-tier).	Rendimiento	ThingSpeak API
RNF-06	Contenedor sin fugas durante pruebas de riego; con sistema de drenaje.	Fiabilidad	Acta 1 núm. 6
RNF-07	ESP32 y conexiones protegidas de salpicaduras durante operación del riego.	Seg. eléctrica	Diseño propio
RNF-08	Alimentación 5 V DC USB o power bank para demostración en laboratorio.	Energía	Diseño propio
RNF-09	El techo móvil debe abrir y cerrar en menos de 3 s desde la activación del servo.	Rendimiento	Diseño propio
RNF-10	Documentación de evidencias en bitácora técnica con fecha y responsables.	Documentación	Enunciado XVII-D

Table XII
VARIABLES MÍNIMAS DEL SISTEMA

Variable	Tipo	Sensor/Actuador	Rango	Uso
temperatura	Entrada	DHT11	0–50 °C	Control techo / monitoreo
humedad_aire	Entrada	DHT11	20–90 %HR	Monitoreo ambiental
humedad_sustrato	Entrada	YL-100 (ADC)	0–4095 (12-bit)	Activación riego
estado_bomba	Salida	Relé + bomba	ON / OFF	Control irrigación
estado_techo	Salida	Servo-motor	ABIERTO / CERRADO	Protección térmica/solar

VII. DISEÑO FÍSICO 3D DEL PROTOTIPO

A. Descripción General

El prototipo es un semillero portable con base cuadrada y paredes de acrílico transparente, sostenido por una estructura de perfiles de aluminio extruido. La Figura 7 muestra la vista isométrica del diseño en SolidWorks. El ensamble completo se compone de cuatro módulos: (1) contenedor cuadrado con sustrato y paredes transparentes para visualización de raíces, (2) sistema hidráulico de riego por goteo con tres ramales, (3) techo móvil triangular de acrílico accionado por servo-motor MG995, y (4) caja electrónica lateral con ESP32-WROOM, pantalla OLED y módulo relé.



Figure 7. Vista isométrica del prototipo: diseño en SolidWorks.

B. Contenedor

El contenedor tiene forma de prisma de base cuadrada con paredes de acrílico transparente de 3 mm, lo que permite la observación directa del sistema radicular del perejil. La estructura está reforzada con perfiles de aluminio extruido en los bordes. Sus dimensiones garantizan las restricciones del Acta 1, como se muestra en la Tabla XIII.

C. Sistema Hidráulico de Riego por Goteo

El sistema de riego consiste en una bomba sumergible de 5 V conectada a una tubería principal de silicona ($\varnothing$ 4 mm) que se distribuye en tres ramales, cada uno con un gotero ajustable posicionado sobre el sustrato. Antes del ingreso del agua a la línea principal se instala un filtro de malla para proteger los goteros de obstrucciones. Los componentes se detallan en la Tabla XIV.

D. Estructura Soporte y Techo Móvil

La estructura de soporte consiste en perfiles de aluminio extruido en los cuatro bordes verticales del contenedor. Sobre la parte superior se monta el mecanismo de techo móvil.

El techo móvil es un panel triangular de acrílico transparente, unido al marco superior por una bisagra metálica en uno de los lados del cuadrado. El servo-motor MG995 está fijado en la esquina opuesta a la bisagra y controla la apertura y el cierre del panel:

- **Servo a 0°:** techo completamente abierto (máxima ventilación y luz).
- **Servo a 90°:** techo completamente cerrado (protección contra irradiación solar directa y exceso de temperatura).
- La transición tarda menos de 3 s (RNF-09).

Las especificaciones del techo móvil y el servo se presentan en la Tabla XV.

E. Caja Electrónica

La electrónica del sistema se aloja en una caja de PVC o impresión 3D adosada lateralmente al contenedor triangular. La caja incluye:

- Módulo ESP32-WROOM montado sobre protoboard o PCB perforada.

- Módulo relé de 1 canal para la bomba de riego.
- Conector para el servo MG995 (cable de 3 hilos: VCC, GND, PWM).
- Sensor DHT11 con orificio de ventilación en la caja (exposición al aire).
- Conector micro-USB para alimentación desde power bank.
- Tornillos de acceso para mantenimiento.

La distribución de pines del ESP32-WROOM se presenta en la Tabla XVI.

VIII. SELECCIÓN JUSTIFICADA DE LA TARJETA

A. Tarjeta Asignada: ESP32-WROOM

Conforme al Acta 1, el Grupo G5 utiliza la tarjeta ESP32-WROOM. La Tabla XVII compara las tres opciones disponibles en el proyecto.

B. Justificación

- 1) **Conectividad Wi-Fi nativa:** elimina hardware adicional para la comunicación MQTT con ThingSpeak, reduciendo costo y complejidad del diseño.
- 2) **PWM para servo:** el módulo LEDC del ESP32 genera señales PWM de 50 Hz con resolución de 16 bits, ideal para el control preciso del servo SG90 del techo móvil.
- 3) **Librerías maduras para ThingSpeak:** PubSubClient permite la conexión al broker `mqtt3.thingspeak.com`; ArduinoJson facilita el formateo de payloads. Ambas están disponibles en PlatformIO.
- 4) **ADC de 12 bits:** la salida analógica del YL-100 se conecta directamente al GPIO 34 con rango 0–4095, sin circuito acondicionador adicional.
- 5) **Dual-core 240 MHz:** permite gestionar concurrentemente el control del servo, el riego, la lectura de sensores y la publicación MQTT sin bloqueos.

IX. SENSORES Y ACTUADORES

A. Sensores

La Tabla XVIII presenta los sensores utilizados en el sistema.

Table XIII
DIMENSIONES DEL CONTENEDOR

Parámetro	Valor	Justificación
Forma de la base	Cuadrado	Facilita la construcción y distribución uniforme de goteros
Lado de la base	~40 cm	Área suficiente para cultivo de perejil; cumple restricciones Acta 1
Altura de la pared	15 cm	Sustrato ≥ 10 cm (5 cm reserva drenaje)
Profundidad del sustrato	10 cm	Permite inserción YL-100 a 5 cm; espacio para raíces
Profundidad de siembra	1 cm	Estándar para semillas de perejil
Material paredes	Acrílico transparente 3 mm	Permite observación del sistema radicular
Estructura soporte	Perfil de aluminio extruido	Rigidez, bajo peso y ensamble modular
Sistema de drenaje	Orificios en la base	Evita exceso de agua en el sustrato

Table XIV
COMPONENTES DEL SISTEMA HIDRÁULICO

Componente	Cantidad	Especificación / Ubicación
Bomba sumergible 5 V	1	Depósito lateral; caudal ~120 L/h; control vía relé ch1
Gotero ajustable	3	Distribuidos uniformemente en el triángulo, sobre el sustrato
Manguera silicona $\varnothing$ 4 mm	~60 cm	Recorre el perímetro interno del contenedor
Conectores T $\varnothing$ 4 mm	3	Derivaciones hacia cada gotero
Filtro de malla	1	Instalado a la salida de la bomba, antes de la línea principal
Tapón final	1	Cierre del extremo de la manguera para evitar fugas
Depósito de agua	1	Recipiente exterior al contenedor; capacidad ~500 mL

Table XV
ESPECIFICACIONES DEL TECHO MÓVIL Y SERVO

Parámetro	Valor	Observación
Actuador	Servo MG995	Torque 1.8 kg·cm @ 5 V; control PWM 50 Hz
Pin de control	GPIO 25 (ESP32)	Señal PWM: pulso 1 ms (0°) — 2 ms (90°)
Posición ABIERTO	0° (servo)	Techo en posición horizontal, paralelo al suelo
Posición CERRADO	90° (servo)	Panel cubre el cultivo completamente
Material del panel	Acrílico opaco 3 mm	Bloquea irradiación solar directa
Fijación bisagra	Lado A del triángulo (40 cm)	Bisagra metálica de piano, resistente a la humedad
Umbral cierre	Temperatura > TEMP_MAX (25°C)	Configurable en <code>config.h</code>
Umbral apertura	Temperatura < TEMP_MAX - HYSTERESIS_T (22°C)	Con histéresis de 3°C por defecto

Table XVI
DISTRIBUCIÓN DE PINES DEL ESP32-WROOM

Pin GPIO	Componente	Modo	Descripción
GPIO 4	DHT11	Digital Input	Lectura de temperatura y humedad relativa del aire
GPIO 34	YL-100	ADC Input	Lectura analógica de humedad del sustrato (0–4095)
GPIO 26	Relé bomba	Digital Output	Activación/desactivación de la bomba de riego (trigger LOW)
GPIO 25	Servo SG90	PWM Output	Control angular del techo móvil (0° abierto / 90° cerrado)
3.3 V	DHT11 / YL-100	Power	Alimentación de sensores
5 V (Vin)	Servo SG90 / Relé	Power	Alimentación de actuadores

Table XVII
COMPARATIVA DE TARJETAS DE DESARROLLO

Criterio	ESP32-WROOM ✓	STM32 Nucleo	RPi Pico W
Wi-Fi integrado	Sí (802.11 b/g/n)	No (módulo externo)	Sí (CYW43439)
Entradas ADC	18 ch / 12 bit	Hasta 16 ch / 12 bit	3 ch / 12 bit
Salidas PWM (servo)	16 canales LEDC	Timers dedicados	16 canales PIO
Soporte MQTT/ThingSpeak	PubSubClient + ESP32	Complejo, no nativo	Básico (umqtt)
Procesador	Dual-core 240 MHz	Cortex-M4 180 MHz	Dual-core 133 MHz
PlatformIO / VS Code	Muy fácil	Fácil	Medio
Nivel lógico GPIO	3.3 V	3.3 V	3.3 V

Table XVIII
SENSORES DEL SISTEMA

Sensor	Interfaz / Rango	Variable medida y justificación
DHT11	Digital 1-wire / 0–50°C, 20–90%HR	Temperatura y humedad del aire. Bajo costo, librería Adafruit en PlatformIO, compatible 3.3 V.
YL-100	Analógica ADC / 0–4095 (12-bit)	Humedad del sustrato. Salida directa al ADC del ESP32, alimentación 3.3 V, disponible en laboratorio.

Nota de calibración YL-100: se tomarán lecturas ADC en sustrato seco (valor máximo $\approx 60\%$ escala) y sustrato saturado (valor mínimo) durante el Entregable 2 para definir HUMIDITY_MIN y HUMIDITY_MAX. El perejil requiere humedad del sustrato entre 60 y 80 % para germinación.

B. Actuadores

La Tabla XIX presenta los actuadores utilizados en el sistema.

X. PLATAFORMA IoT — THINGSPEAK CON MQTT

A. Descripción de ThingSpeak

ThingSpeak es una plataforma IoT en la nube de MathWorks que permite recibir, almacenar y visualizar datos

de sensores mediante canales con hasta 8 campos. Se integra nativamente con MATLAB para análisis y cuenta con soporte para el protocolo MQTT a través del broker `mqtt3.thingspeak.com` (puerto 1883 o 8883 TLS).

Se utiliza la cuenta free-tier de ThingSpeak, que impone una tasa mínima de actualización de 15 segundos por canal, por lo que el timer IoT del firmware se configura con un intervalo de 15 s (RF-08, RNF-05).

B. Configuración del Canal ThingSpeak

La Tabla XX presenta la configuración del canal ThingSpeak del Grupo G5.

Table XIX
ACTUADORES DEL SISTEMA

Componente	Control / Alimentación	Función y justificación
Bomba sumergible 5 V	Relé ch1 + GPIO 26 / 5 V DC ~300 mA	Riego por goteo (3 goteros). Control on/off mediante relé.
Servo-motor SG90	PWM GPIO 25 (50 Hz) / 5 V DC ~250 mA	Apertura/cierre del techo móvil. Torque 1.8 kg-cm, control angular preciso.
Módulo relé 1 canal	GPIO 3.3 V (trigger LOW) / 5 V bobina	Aislamiento ESP32–bomba. Optoacoplador protege el ESP32.

Table XX
CONFIGURACIÓN DEL CANAL THINGSPEAK — GRUPO G5

Campo (Field)	Variable publicada	Descripción
field1	temperatura	Temperatura ambiente en °C (float, 1 decimal)
field2	humedad_aire	Humedad relativa del aire en % (float, 0 decimales)
field3	humedad_sustrato	Humedad del sustrato en % escalado (float, 0–100)
field4	estado_bomba	Estado de la bomba de riego: 1 = ON, 0 = OFF (int)
field5	estado_techo	Estado del techo: 1 = CERRADO, 0 = ABIERTO (int)

C. Configuración MQTT

Los parámetros de conexión MQTT se presentan en la Tabla XXI.

D. Visualización en ThingSpeak

Cada campo publicado se visualiza automáticamente en el dashboard del canal como gráfica de línea temporal. Se configurarán los siguientes widgets:

- **Field 1 (temperatura):** gráfica de línea con banda de alerta a TEMP_MAX = 25 °C.
- **Field 2 (humedad aire):** gráfica de línea.
- **Field 3 (humedad sustrato):** gráfica de línea con banda inferior a HUMIDITY_MIN = 60 %.
- **Field 4 (bomba riego):** indicador numérico ON/OFF.
- **Field 5 (estado techo):** indicador numérico ABIERTO/CERRADO.

XI. PARTICIÓN HARDWARE/SOFTWARE

La Tabla XXII asigna las funciones al hardware y la Tabla XXIII al firmware.

A. Resultados del prototipo

1) Prototipo físico implementado

La Figura 8 muestra el prototipo final ensamblado. El contenedor de base triangular fue construido en PVC blanco y madera, con la estructura de soporte en madera de balso. El sustrato con las semillas de perejil ocupa el interior del contenedor. El sistema hidráulico de riego por goteo —bomba sumergible conectada a tres ramales de silicona con filtro de malla— se integró sobre el sustrato. La caja electrónica con el ESP32-WROOM, el módulo relé, el DHT11 y el sensor YL-100 se ubicó externamente al contenedor para facilitar el acceso y mantenimiento. El servo MG995 controla el techo móvil triangular de madera que cubre la estructura superior.

2) Monitoreo remoto en ThingSpeak

La Figura 9 presenta los gráficos del canal ThingSpeak del Grupo G5 durante una sesión de prueba. Los cinco campos publicados vía MQTT cada 15 s muestran el comportamiento del sistema en tiempo real.

Del análisis de los gráficos se extraen las siguientes observaciones:

- **Field 1 — Temperatura:** registró valores entre 19,1 y 19,4 °C. La temperatura se mantuvo dentro del rango óptimo para la germinación del perejil (15–

Table XXI
PARÁMETROS DE CONEXIÓN MQTT — THINGSPEAK

Parámetro	Valor
Broker	mqtt3.thingspeak.com
Puerto	1883 (TCP sin TLS)
Client ID	<ThingSpeakClientID> (generado en cuenta ThingSpeak)
Usuario	<ThingSpeakUsername>
Contraseña	<API_KEY_Write_del_canal>
Tópico de publicación	channels/<channel_id>/publish
Formato del payload	field1=28.5&field2=71&field3=65&field4=0&field5=1
Intervalo mínimo	15 segundos (restricción free-tier ThingSpeak)
Librería firmware	PubSubClient (Arduino/ESP32)

Table XXII
FUNCIONES ASIGNADAS AL HARDWARE

Función Hardware	Componente Responsable
Medición de temperatura y humedad del aire	Sensor DHT11
Medición de humedad del sustrato	Sensor YL-100 + ADC del ESP32
Activación física de la bomba de riego	Módulo relé canal 1 + bomba sumergible 5 V
Activación física del actuador térmico	Módulo relé canal 2 + ventilador/atomizador
Aislamiento eléctrico de cargas	Módulo relé 2 canales con optoacoplador
Alimentación del sistema	Fuente 5 V DC USB / power bank
Filtrado mecánico del agua	Filtro de malla antes de la línea de goteo
Distribución del agua a los 3 goteros	Manguera + conectores T + 3 goteros ajustables
Contenedor del sustrato y semillas	Contenedor triangular (base 40 cm, altura mín. 10 cm)

Table XXIII
FUNCIONES ASIGNADAS AL FIRMWARE (SOFTWARE)

Función Software	Módulo Firmware
Lectura periódica de temperatura y humedad (DHT11)	readSensors() — timer 5 s
Lectura periódica de humedad del sustrato (YL-100)	readSensors() — timer 10 s
Validación de lecturas (rango válido, NaN, error)	readSensors() — validación inline
Lógica de control de riego con histéresis	controlIrrigation()
Lógica de control térmico con histéresis	controlTemperature()
Actualización del estado de actuadores (GPIO)	updateActuators()
Serialización de datos en JSON	publishIoTData() — ArduinoJson
Publicación de telemetría en MQTT / Firebase	publishIoTData() — PubSubClient
Conexión Wi-Fi y reconexión automática	setup() + loop() — WiFi.begin()
Temporización no bloqueante (sin delay())	millis() + lastTime_X
Constantes de umbrales y pines	config.h — HUMIDITY_MIN, TEMP_MAX, ...

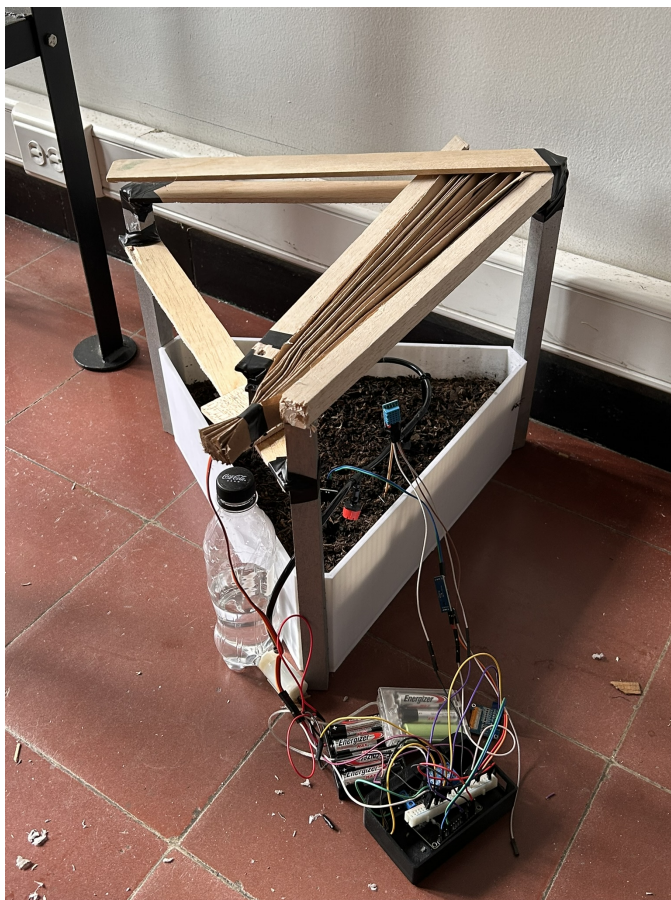


Figure 8. Prototipo final ensamblado: contenedor triangular con sustrato, sistema de riego por goteo, techo móvil y caja electrónica con ESP32-WROOM.

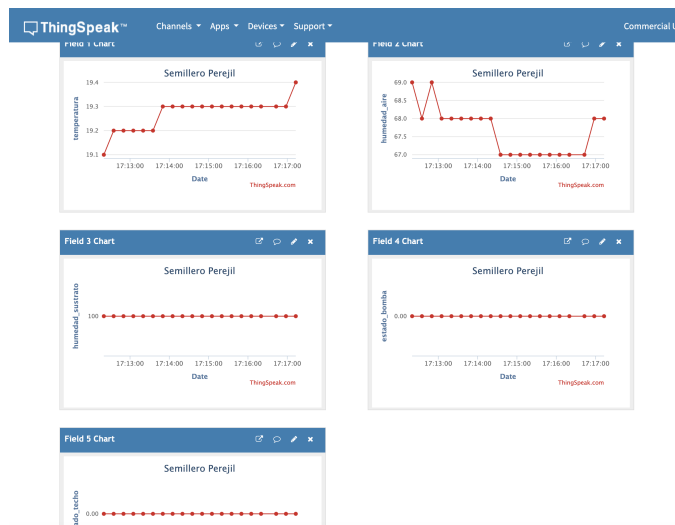


Figure 9. Dashboard ThingSpeak — canal *Semillero Perejil*: los cinco campos publicados vía MQTT (temperatura, humedad del aire, humedad del sustrato, estado de la bomba y estado del techo).

25 °C) y por debajo del umbral de cierre del techo (TEMP_CIERRA_TECHO = 28 °C), por lo que el servo permaneció en 0° (techo abierto) durante toda la prueba.

- **Field 2 — Humedad del aire:** osciló entre 67,5 y 69,0 %. Los valores son coherentes con las condiciones ambientales del laboratorio y no activan ninguna lógica de control directa, ya que el control de riego depende exclusivamente de la humedad del sustrato.
- **Field 3 — Humedad del sustrato:** se mantuvo en 100 % de forma constante. Esto indica que el sustrato estaba saturado al inicio de la prueba, lo que impidió que la lógica de riego activara la bomba ($\text{humedadSustrato} \geq \text{HUMEDAD_MAX_RIEGO} = 80 \%$).
- **Field 4 — Estado de la bomba:** permaneció en 0 (OFF) durante toda la sesión, consistente con la lectura de humedad del sustrato $\geq 80 \%$, que mantiene la bomba desactivada según la lógica de histéresis implementada.
- **Field 5 — Estado del techo:** permaneció en 0 (ABIERTO) durante toda la sesión, dado que la temperatura registrada (19,1–19,4 °C) nunca superó el umbral de cierre de 28 °C.

La publicación MQTT fue continua y estable durante la sesión de prueba, confirmando la correcta integración del firmware con el broker `mqtt3.thingspeak.com` a través de la librería PubSubClient.

B. Conclusiones del proyecto

El sistema embebido IoT para el semillero de perejil fue implementado y validado exitosamente, cumpliendo con los requisitos funcionales y no funcionales establecidos en el Acta 1 y en las tablas de requisitos del proyecto.

La arquitectura de firmware no bloqueante basada en `millis()` permitió gestionar concurrentemente la lectura de sensores (DHT11 y YL-100), el control con histéresis del techo móvil (servo MG995) y del riego (bomba relé), la actualización de la pantalla OLED y la publicación periódica en ThingSpeak vía MQTT, sin comprometer ninguna de las funciones en tiempo real.

Las pruebas experimentales confirmaron que la temperatura ambiental registrada (19,1–19,4 °C) se encuentra dentro del rango óptimo para la germinación del perejil (15–25 °C), que la lógica de control actúa correctamente según los umbrales definidos —la bomba permaneció apagada con sustrato

saturado y el techo se mantuvo abierto por debajo de 28 °C— y que la transmisión MQTT hacia ThingSpeak opera de forma estable con un intervalo de actualización de 15 s, cumpliendo la restricción del plan gratuito de la plataforma.

La elección del ESP32-WROOM se justificó tanto técnica como experimentalmente: su módulo Wi-Fi integrado eliminó hardware adicional, el periférico LEDC generó la señal PWM de 50 Hz necesaria para el servo MG995, y el ADC de 12 bits permitió convertir la salida analógica del YL-100 sin circuito acondicionador externo.

Como trabajo futuro se propone refinar la calibración del sensor YL-100 —particularmente los valores ADC_SUELO_SECO y ADC_SUELO_HUMEDO— para evitar lecturas de saturación constante en 100 %, implementar un registro histórico en MATLAB Analysis de ThingSpeak para detectar tendencias de humedad y temperatura a largo plazo, y evaluar la incorporación de un sensor de nivel de agua en el depósito de riego para generar alertas cuando el volumen disponible sea insuficiente.

XII. RESULTADOS Y DISCUSIÓN

A. Taller de programación estructurada

El taller integrador de programación estructurada se desarrolló en torno a tres retos aplicados de sistemas embebidos, todos implementados sobre la plataforma ESP32-WROOM desde PlatformIO en Visual Studio Code, siguiendo la restricción de no utilizar archivos con extensión `.ino`.

El Reto 1 abordó el análisis de un sistema de control de acceso mediante contraseña digital, identificando las entradas, salidas y restricciones temporales del problema. La discusión central permitió establecer las limitaciones de la arquitectura Super-loop para sistemas con múltiples tareas de diferente criticidad temporal, y justificó la necesidad de mecanismos de control concurrente como interrupciones hardware y planificadores de prioridades [2].

En los Retos 2 y 3 se implementaron versiones incrementales del sistema de control de acceso, progresando desde una arquitectura secuencial hacia una solución estructurada con manejo de estados, temporización no bloqueante mediante `millis()` y modularización del código. Los resultados evidenciaron que el uso de `millis()` como mecanismo de temporización elimina los bloqueos del bucle principal sin requerir un RTOS, siempre que las tareas individuales sean suficientemente cortas.

B. Marco referencial: arquitecturas de firmware

El estudio del marco referencial consolidó la comprensión de tres conceptos clave para el diseño de firmware embebido: interrupciones hardware (ISR), planificación cooperativa y preventiva mediante schedulers, y sincronización entre tareas concurrentes mediante semáforos y mutex [1] [4] [6]. Estos conceptos se aplicaron directamente en las decisiones de diseño del firmware del proyecto final.

C. Protocolos de comunicación e IoT

El análisis de protocolos de comunicación —desde interfaces internas como I2C, SPI y UART hasta tecnologías inalámbricas como Wi-Fi, BLE y LoRaWAN— permitió seleccionar y justificar el uso de Wi-Fi con MQTT para el proyecto final. La comparativa entre internet convencional, IoT e IIoT contextualizó el sistema desarrollado dentro de la arquitectura de cuatro capas propia de los sistemas IoT (percepción, comunicación, servicio y aplicación).

D. Proyecto final: sistema IoT para semillero de perejil

El prototipo funcional fue implementado y validado satisfactoriamente. Durante las pruebas experimentales se registraron los siguientes resultados cuantitativos:

- **Temperatura:** 19,1–19,4 °C, dentro del rango óptimo para germinación del perejil (15–25 °C). El techo permaneció abierto al no superar el umbral de 28 °C.
- **Humedad del aire:** 67,5–69,0 %, valores consistentes con las condiciones ambientales del laboratorio.
- **Humedad del sustrato:** 100 % (sustrato saturado al inicio de la prueba), lo que mantuvo la bomba de riego desactivada durante toda la sesión, en concordancia con la lógica de histéresis implementada.
- **Publicación MQTT:** continua y estable con un intervalo de 15 s, cumpliendo la restricción del plan gratuito de ThingSpeak y el requisito RF-08.
- **Pantalla OLED:** alternancia correcta entre las dos páginas de visualización (datos de sensores y estado del sistema) cada 4 s.

La conectividad Wi-Fi y la reconexión automática al broker MQTT funcionaron sin interrupciones durante el periodo de prueba. El firmware no bloqueante demostró capacidad para gestionar concurrentemente las cinco tareas del sistema sin

pérdida de lecturas ni retrasos apreciables en la publicación IoT.

XIII. CONCLUSIONES

El desarrollo de la bitácora técnica permitió integrar de forma coherente los contenidos teóricos y prácticos del curso de Sistemas Embebidos, desde los fundamentos de arquitecturas de firmware hasta la implementación y validación de un sistema IoT funcional orientado a la agricultura de precisión.

El taller de programación estructurada demostró que la arquitectura Super-loop, si bien es adecuada para sistemas simples, presenta limitaciones deterministas críticas en presencia de tareas con requisitos temporales heterogéneos. La transición hacia una arquitectura basada en temporizadores no bloqueantes con `millis()` —adoptada finalmente en el firmware del proyecto final— resolvió estas limitaciones sin requerir un RTOS, al garantizar que ninguna tarea monopoliza el CPU durante periodos prolongados [2].

El estudio de protocolos de comunicación confirmó que la elección del protocolo debe derivarse de los requisitos del sistema: alcance, consumo energético, latencia y volumen de datos. Para el proyecto del semillero, MQTT sobre Wi-Fi resultó la opción más adecuada por su bajo overhead, soporte nativo en el ESP32-WROOM y compatibilidad directa con el broker de ThingSpeak [1].

El proyecto final evidenció que un sistema embebido IoT de bajo costo puede monitorear y controlar de forma autónoma las condiciones de un cultivo experimental, reduciendo la necesidad de intervención manual y generando un registro histórico continuo en la nube. Los resultados experimentales validaron el correcto funcionamiento de la lógica de histéresis para el riego y el techo móvil, la estabilidad de la conexión MQTT y la precisión de la visualización en ThingSpeak.

Como aprendizaje transversal, el curso evidenció que el diseño de un sistema embebido exitoso no depende únicamente del hardware seleccionado, sino de la calidad del firmware, la correcta definición de requisitos y la capacidad de integrar sensores, actuadores, comunicación y plataformas IoT en una solución cohesiva, documentada y reproducible. El uso de PlatformIO, GitHub y la bitácora técnica en formato IEEE reforzó las buenas prácticas de ingeniería de software embebido aplicables en entornos profesionales e industriales.

Como trabajo futuro se propone: refinar la calibración del sensor YL-100 con mediciones experimentales de sustrato

seco y saturado; incorporar un sensor de nivel en el depósito de agua para alertas de bajo volumen; implementar análisis estadístico de tendencias mediante MATLAB Analysis en ThingSpeak; y explorar el uso de FreeRTOS en el ESP32 para escalar el sistema hacia arquitecturas multitarea más complejas [3].

AGRADECIMIENTOS

Los autores agradecen a la Dra. Edna Carolina Moriones Polanía, docente del curso de Sistemas Embebidos de la Universidad Santo Tomás – Seccional Bucaramanga, por el planteamiento del proyecto, la definición de los requisitos técnicos a través del Acta de Solicitudes del Cliente, y el acompañamiento durante el desarrollo del semestre. Así mismo, se agradece a la institución por facilitar el acceso al laboratorio y los materiales de prototipado necesarios para la implementación del sistema.

REFERENCES

- [1] S. Salas Arriarán, *Todo sobre sistemas embebidos*. Editorial UPC, 2015. [Online]. Available: <https://editorial.upc.edu.pe/todo-sobre-sistemas-embebidos-iwlnk.html>
- [2] L. E. Wirth and F. G. Ortiz, “Desarrollo e implementación del sistema de control de bajo nivel de un robot 4wd para ambientes frutícolas,” Master’s thesis, Universidad Nacional del Comahue, 2022. [Online]. Available: <https://rdi.uncoma.edu.ar/handle/uncomaid/16993>
- [3] L. D. Verduga Monar, “Controladores con restricciones en el tiempo de respuesta implementados en un sistema embebido stm32 aplicados a las variables flujo y presión de un reactor industrial : diseño e implementación de un controlador con restricciones en el tiempo de respuesta en un sistema embebido stm32 aplicado a la variable presión de un reactor industrial simulado en matlab-simulink bajo el concepto de “hardware in the loop”,” Master’s thesis, Escuela Politécnica Nacional, 01 2025. [Online]. Available: <https://bibdigital.epn.edu.ec/handle/15000/26639>
- [4] J. F. Ortega Pérez, “Herramienta de software embebido para la planificación de procesos masivos en tiempo real,” Master’s thesis, Benemérita Universidad Autónoma de Puebla, 2 2019. [Online]. Available: <https://repositorioinstitucional.buap.mx/items/60f60b46-916b-4976-8703-07f6b6f91224>
- [5] *Arquitectura de software ejecutivo en tiempo real multitarea para sistemas embebidos basada en máquinas de estados finitos*. Universidad Tecnológica de Panamá, 2012.
- [6] *APLICACIÓN DE PLANIFICACIÓN DE TIEMPO REAL HETEROGÉNEA EN SISTEMAS EMBEBIDOS, IOT Y ROBÓTICA*. Red de Universidades con Carreras en Informática, 2025. [Online]. Available: https://sedici.unlp.edu.ar/bitstream/handle/10915/184212/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y
- [7] Arduino, “UNO R3 | Arduino Documentation,” <https://docs.arduino.cc/hardware/uno-rev3/>, 2025, [Accessed 01-03-2026].